

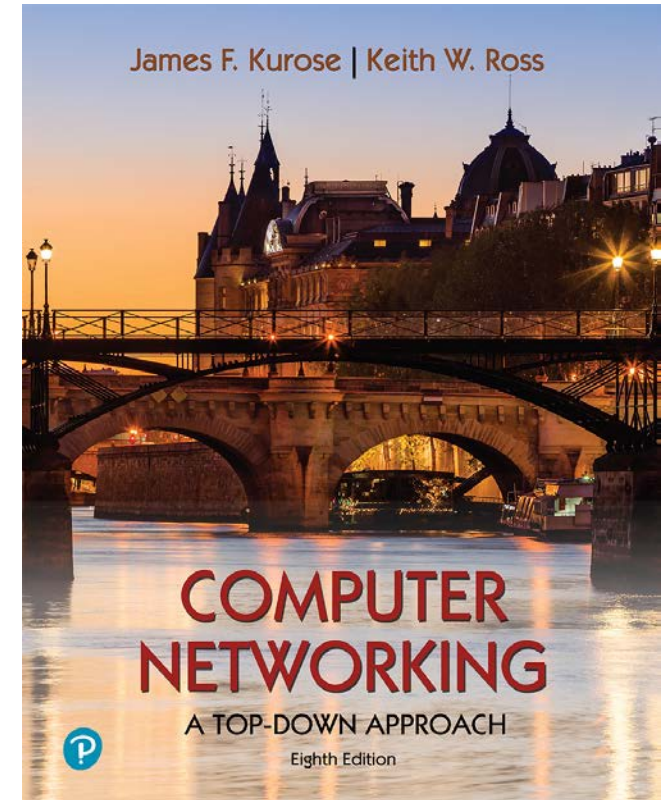
Chapter 3

Transport Layer

Dr. Qixia Zhang

Department of Computer Science
UiT The Arctic University of Norway

All material copyright 1996-2023
J.F Kurose and K.W. Ross, All Rights Reserved



*Computer Networking: A
Top-Down Approach*

8th edition

Jim Kurose, Keith Ross
Pearson, 2020

Chapter 3: roadmap

- Transport-layer services
- Multiplexing and demultiplexing
- Connectionless transport: UDP
- **Principles of reliable data transfer**
- Connection-oriented transport: TCP
- Principles of congestion control
- TCP congestion control
- Evolution of transport-layer functionality



Two principal Internet transport protocols

■ **TCP:** Transmission Control Protocol

- reliable, in-order delivery
- congestion control
- flow control
- connection setup

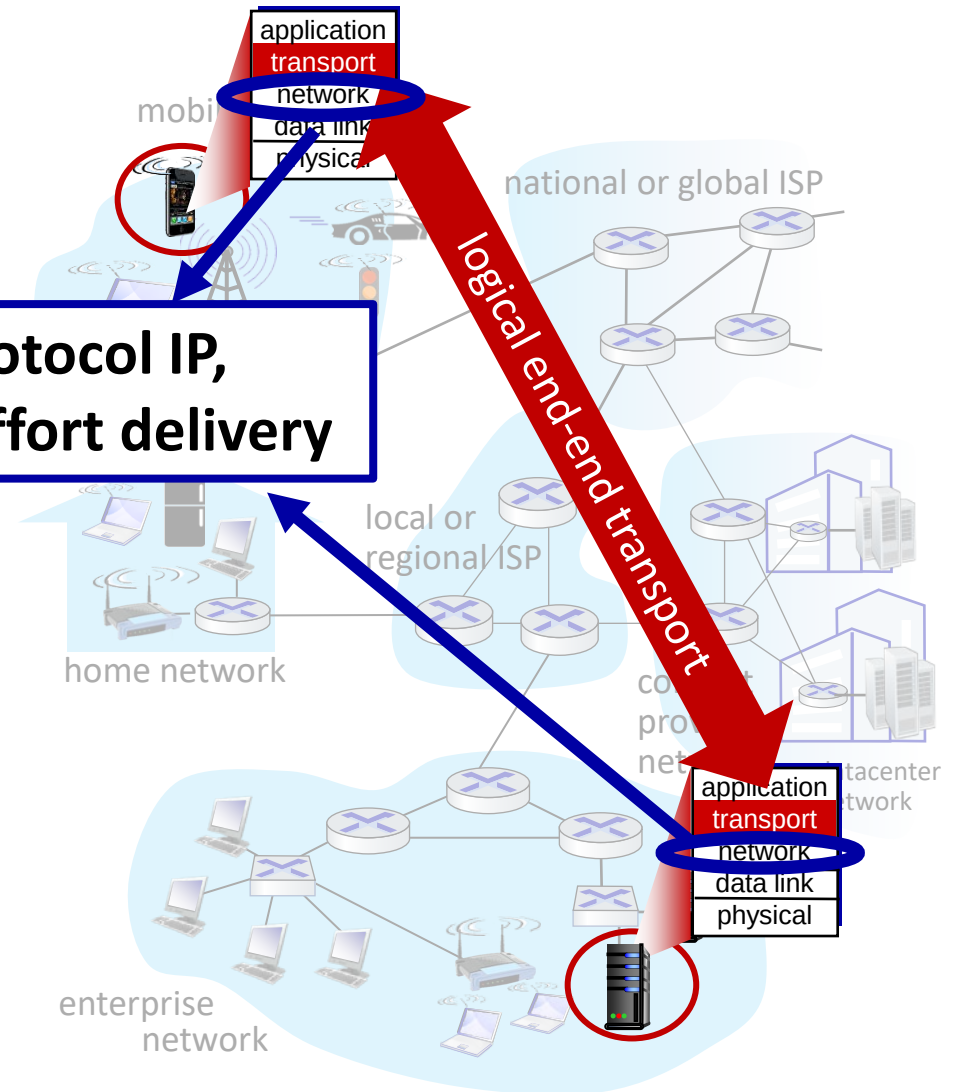
■ **UDP:** User Datagram Protocol

- unreliable, unordered delivery
- simple extension of “best-effort” IP

■ services *not* available:

- delay guarantees
- bandwidth guarantees

***The internet protocol IP,
really is a best-effort delivery**



UDP: User Datagram Protocol

- “no frills,” “bare bones” Internet transport protocol
- “best effort” service, UDP segments may be:
 - lost
 - delivered out-of-order to app
- *connectionless*:
 - no handshaking between UDP sender, receiver
 - each UDP segment handled independently of others

Why is there a UDP?

- no connection establishment (which can add RTT delay)
- simple: no connection state at sender, receiver
- small header size
- no congestion control
 - UDP can blast away as fast as desired!
 - can function in the face of congestion

UDP: User Datagram Protocol

- UDP use cases:
 - streaming multimedia apps (loss tolerant, rate sensitive)
 - DNS (Domain Name System)
 - SNMP (Simple Network Management Protocol)
 - HTTP/3
- if reliable transfer needed over UDP (e.g., HTTP/3):
 - add needed reliability at application layer
 - add congestion control at application layer

Can we implement **reliable data transfer**
in the transport layer? How?

Principles of reliable data transfer

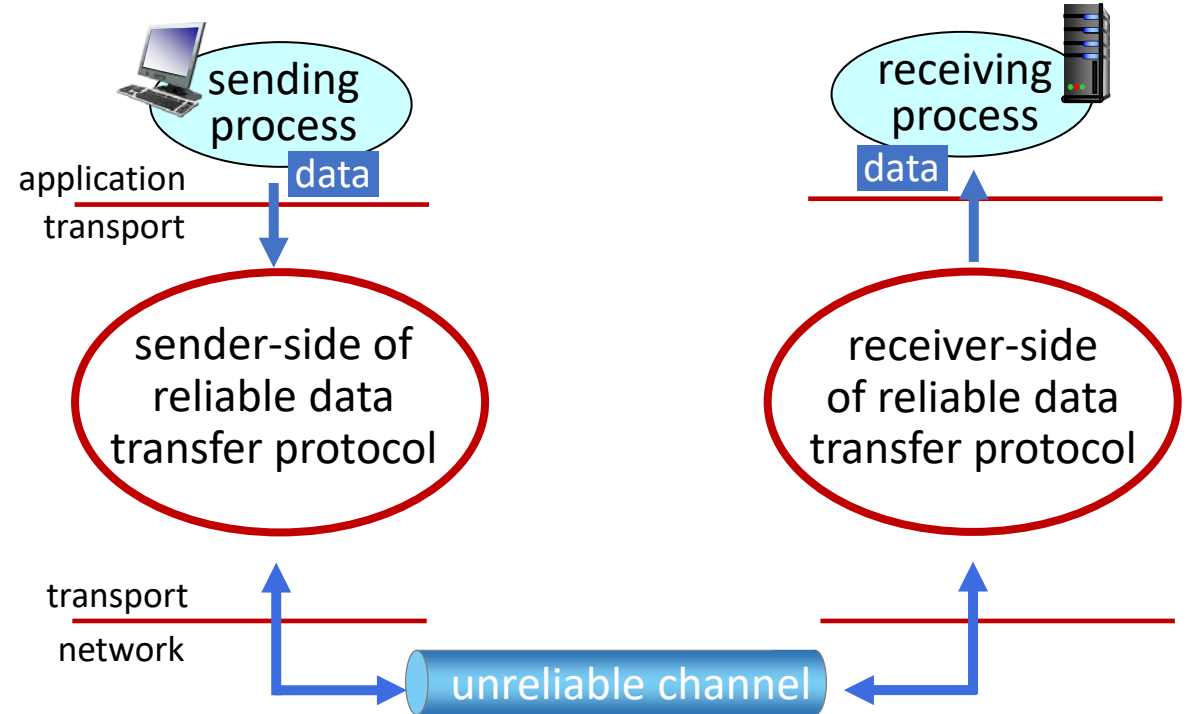
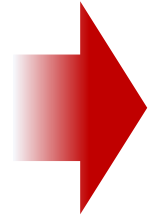


reliable service *abstraction*

Principles of reliable data transfer



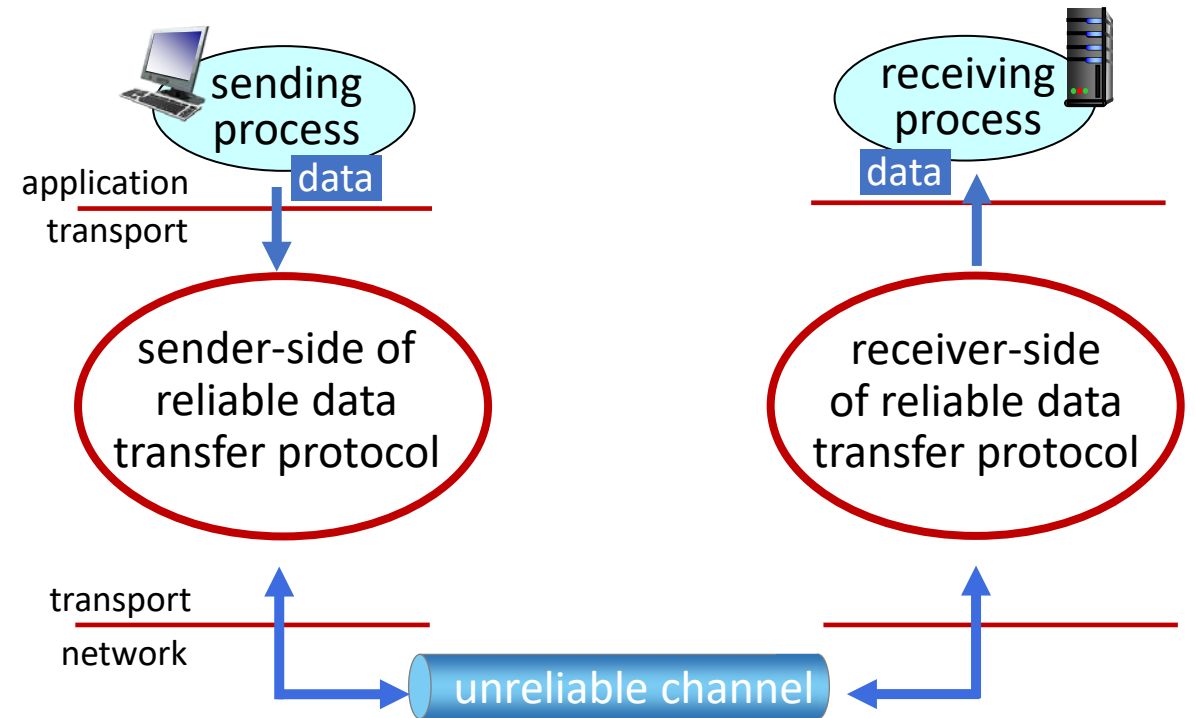
reliable service *abstraction*



reliable service *implementation*

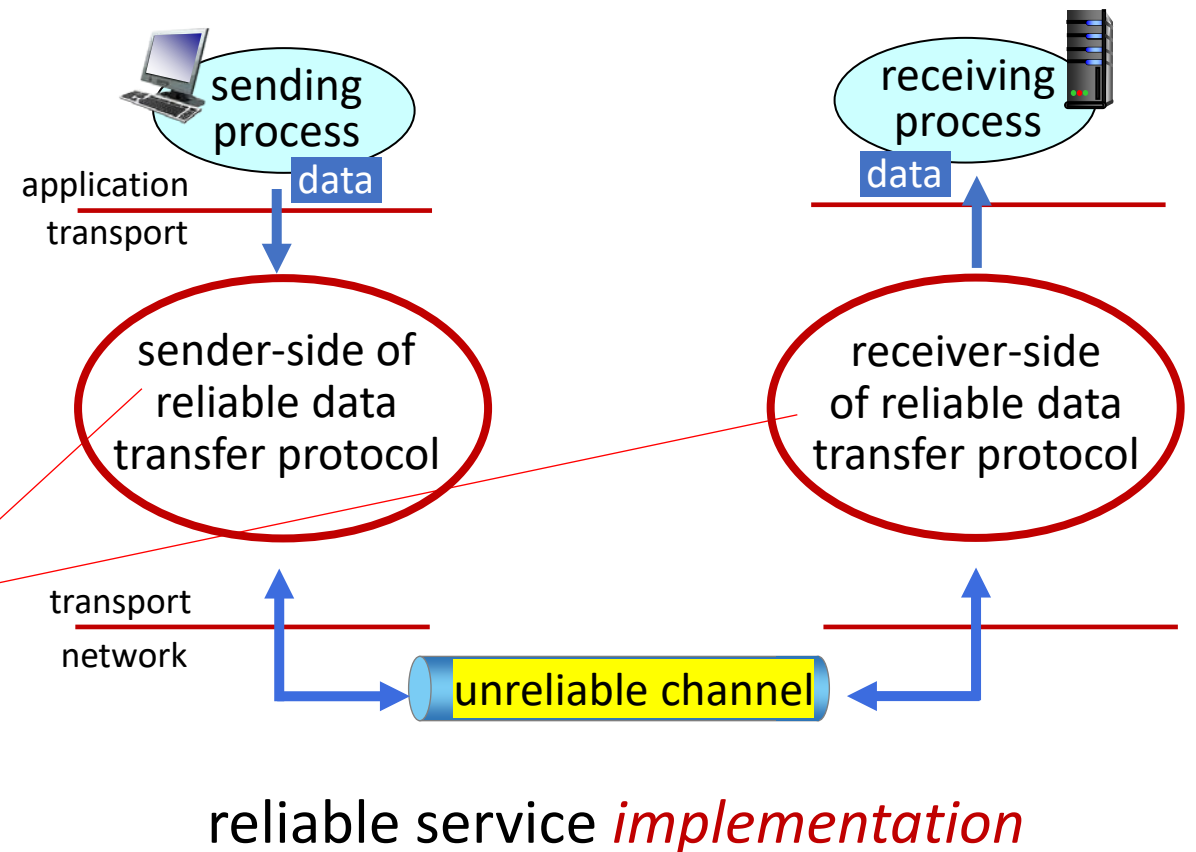
Principles of reliable data transfer

Bi-directional message exchanges between the protocol entities



Principles of reliable data transfer

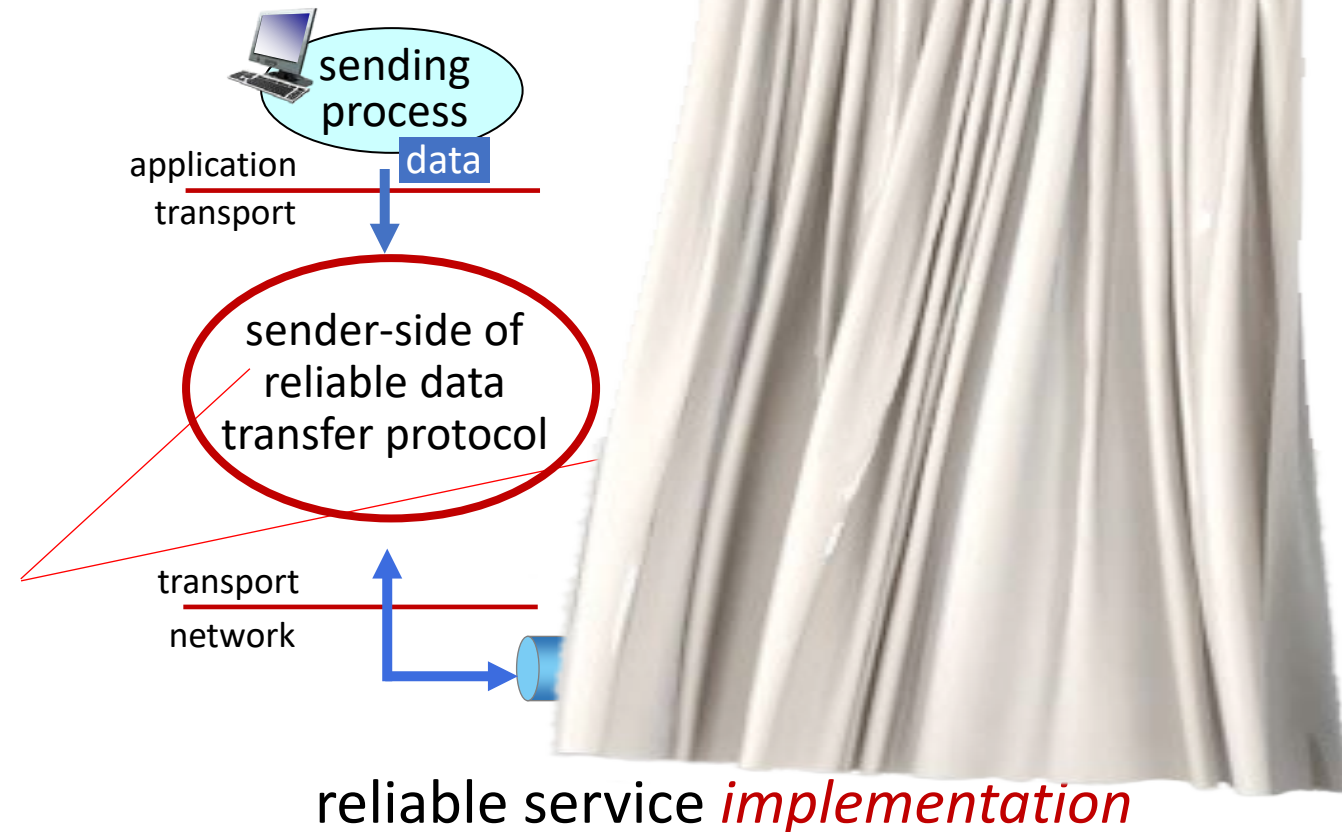
Complexity of reliable data transfer protocol will depend (strongly) on characteristics of **unreliable channel** (lose, corrupt, reorder data?)



Principles of reliable data transfer

Sender, receiver *does not* know the “state” of each other, e.g., was a message received?

- unless communicated via a message



Reliable data transfer: getting started

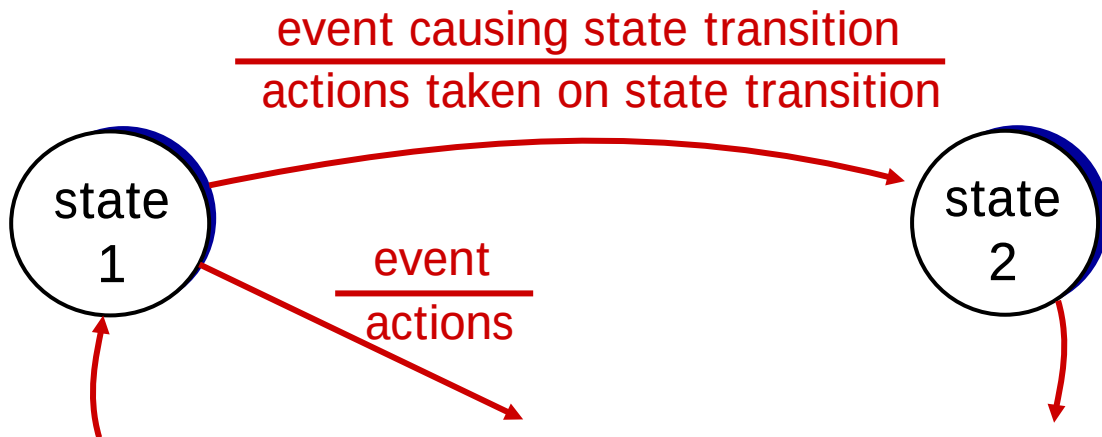
We will:

- incrementally develop sender, receiver sides of reliable data transfer protocol (rdt)
- consider only unidirectional data transfer
 - but control info will flow in both directions!

Reliable data transfer: getting started

We will:

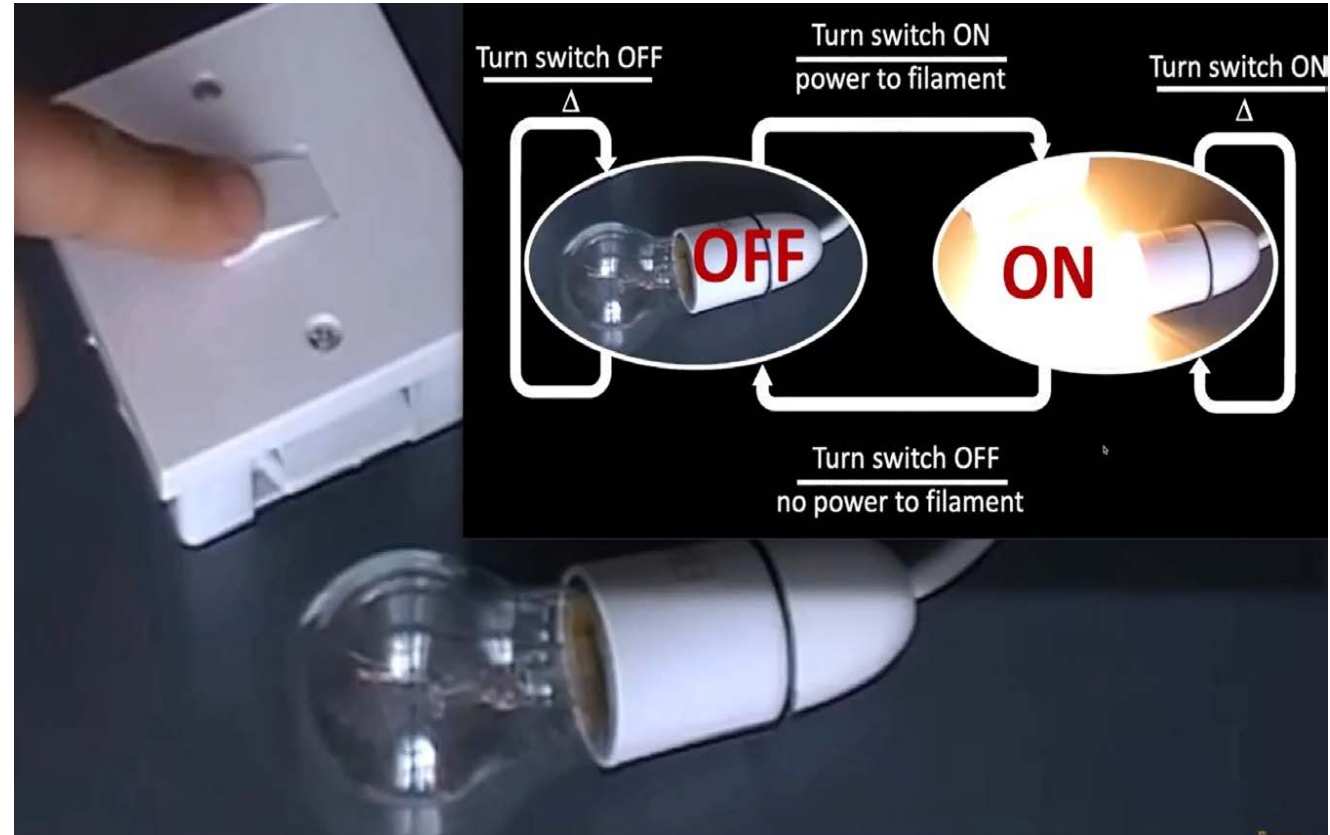
- incrementally develop sender, receiver sides of reliable data transfer protocol (rdt)
- consider only unidirectional data transfer
 - but control info will flow in both directions!
- use finite state machines (FSM) to specify sender, receiver



State = current condition, what the protocol is waiting for
Event = triggers, what triggers the transition
Action = responses, what the protocol does during the transition

Reliable data transfer: real-world analogy

- Real world analogy
 - A light bulb in an on state or an off state
 - Transitions between states
 - Consequence of events taking place
 - Actions take by the system



State = current condition, what the protocol is waiting for

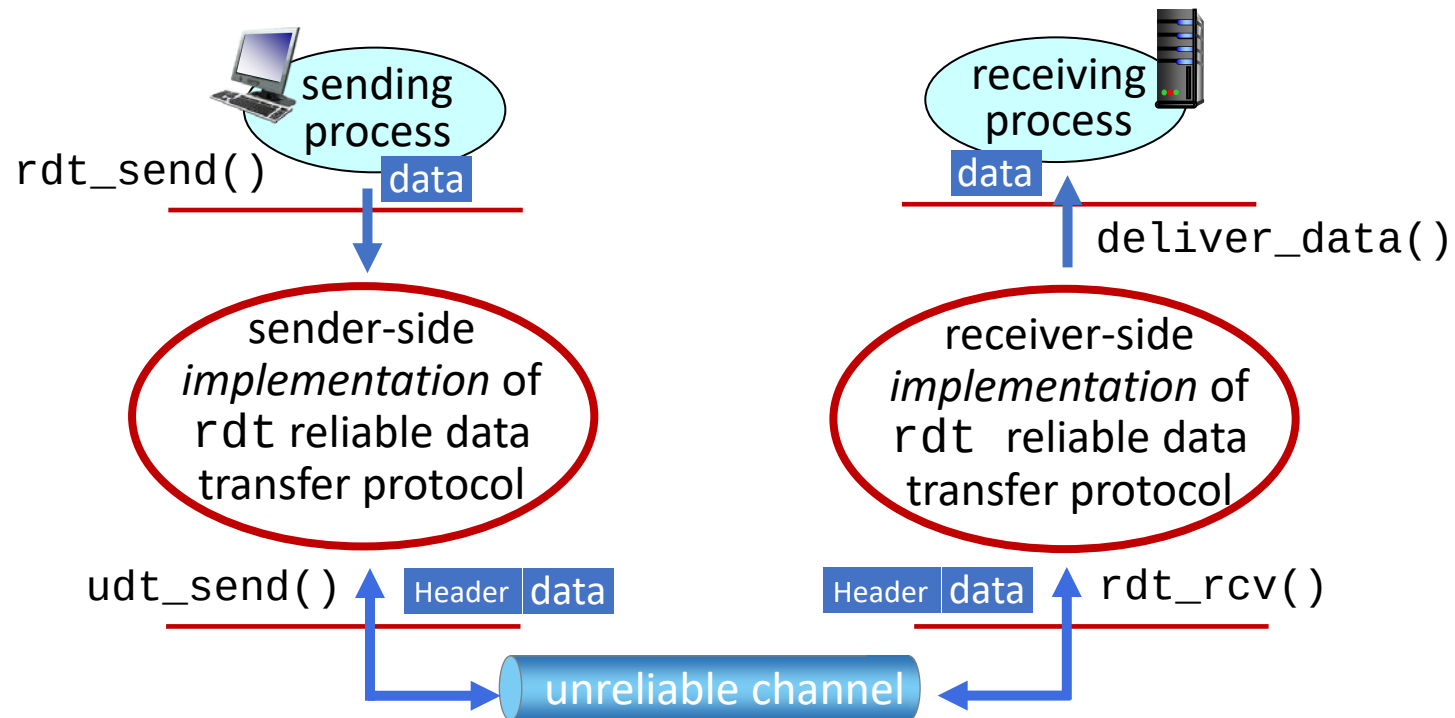
Event = triggers, what happens that triggers transition

Action = responses, what the protocol does during that transition

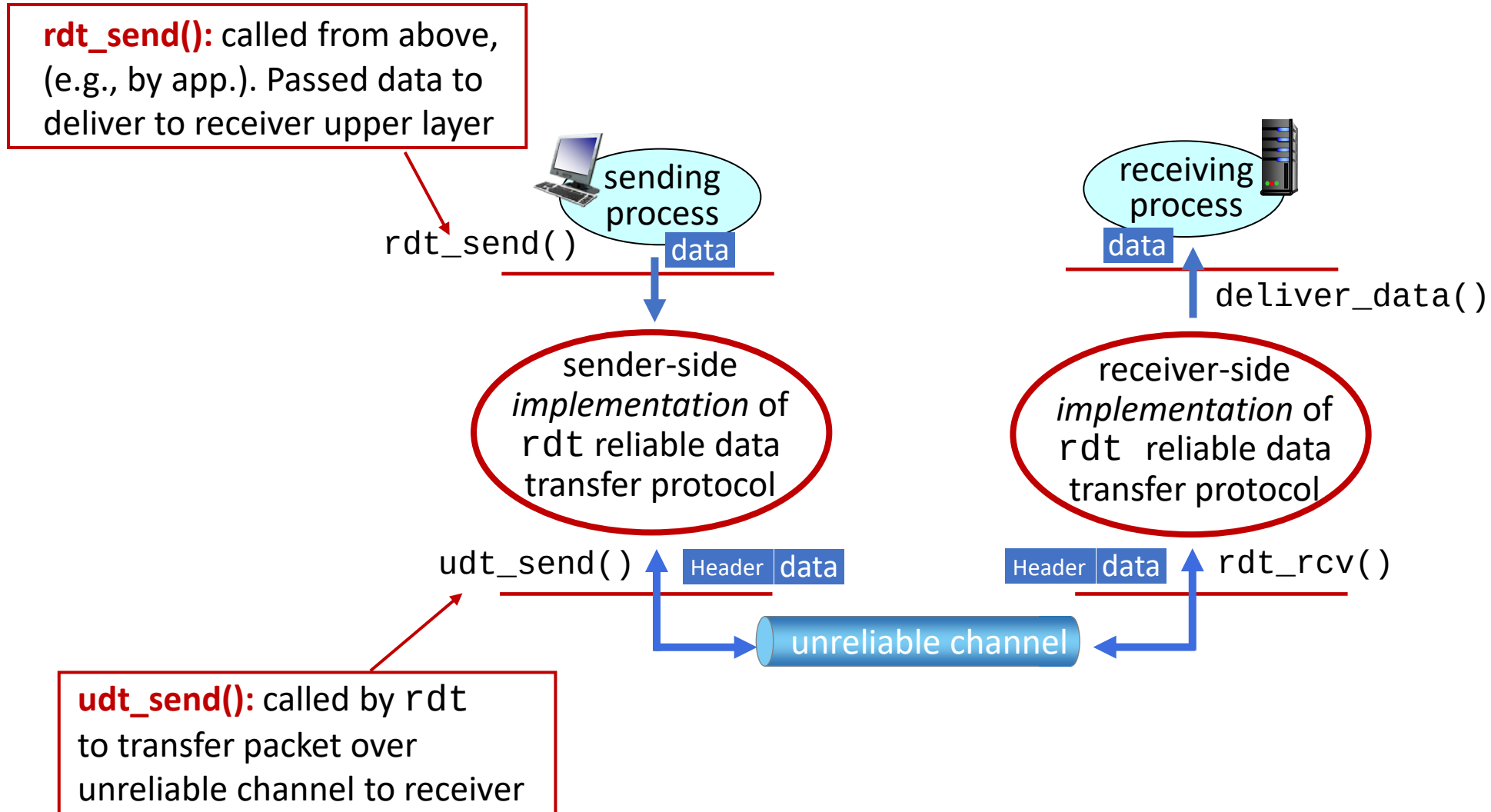
rdt1.0: reliable transfer over a reliable channel

- underlying channel perfectly reliable
 - no bit errors
 - no loss of packets
- *separate* FSMs for sender, receiver:
 - sender sends data into underlying channel
 - receiver reads data from underlying channel

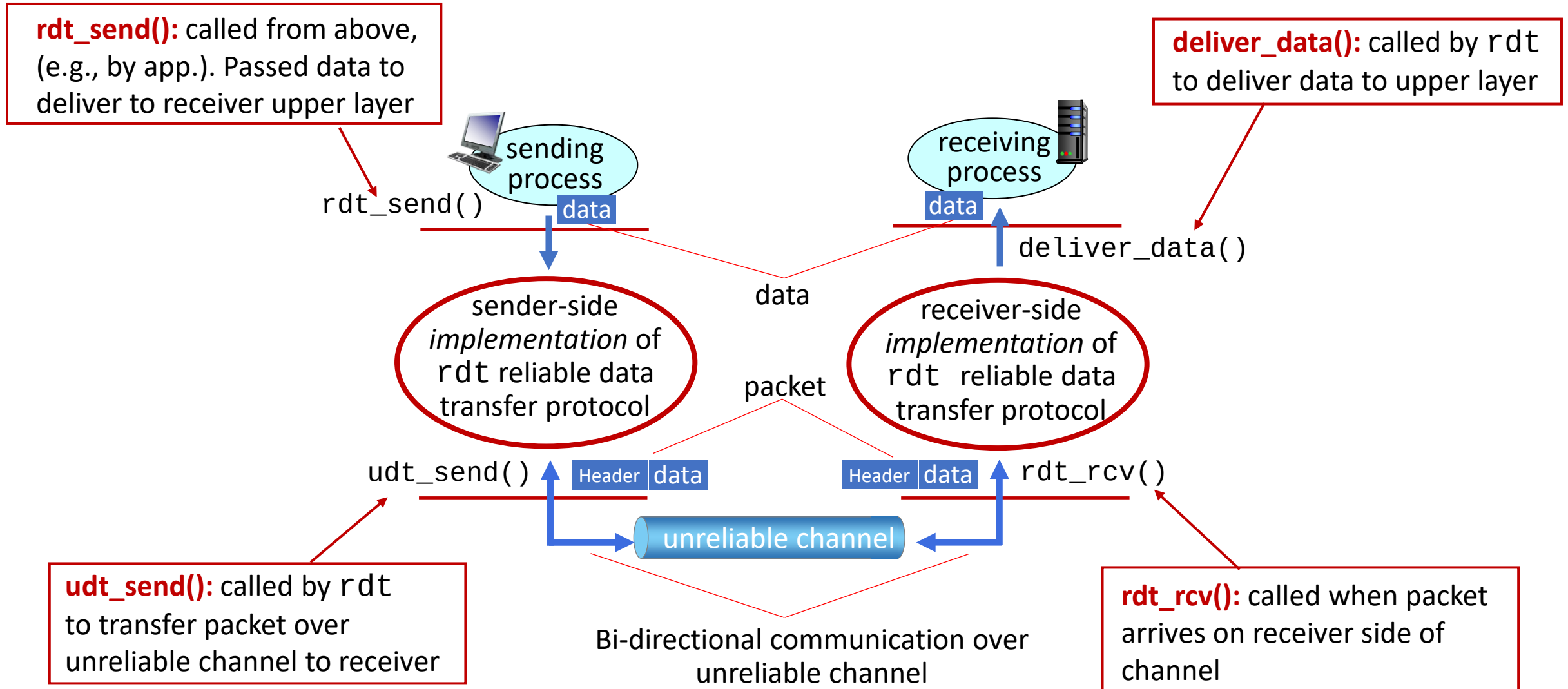
Reliable data transfer protocol (rdt): interfaces



Reliable data transfer protocol (rdt): interfaces



Reliable data transfer protocol (rdt): interfaces



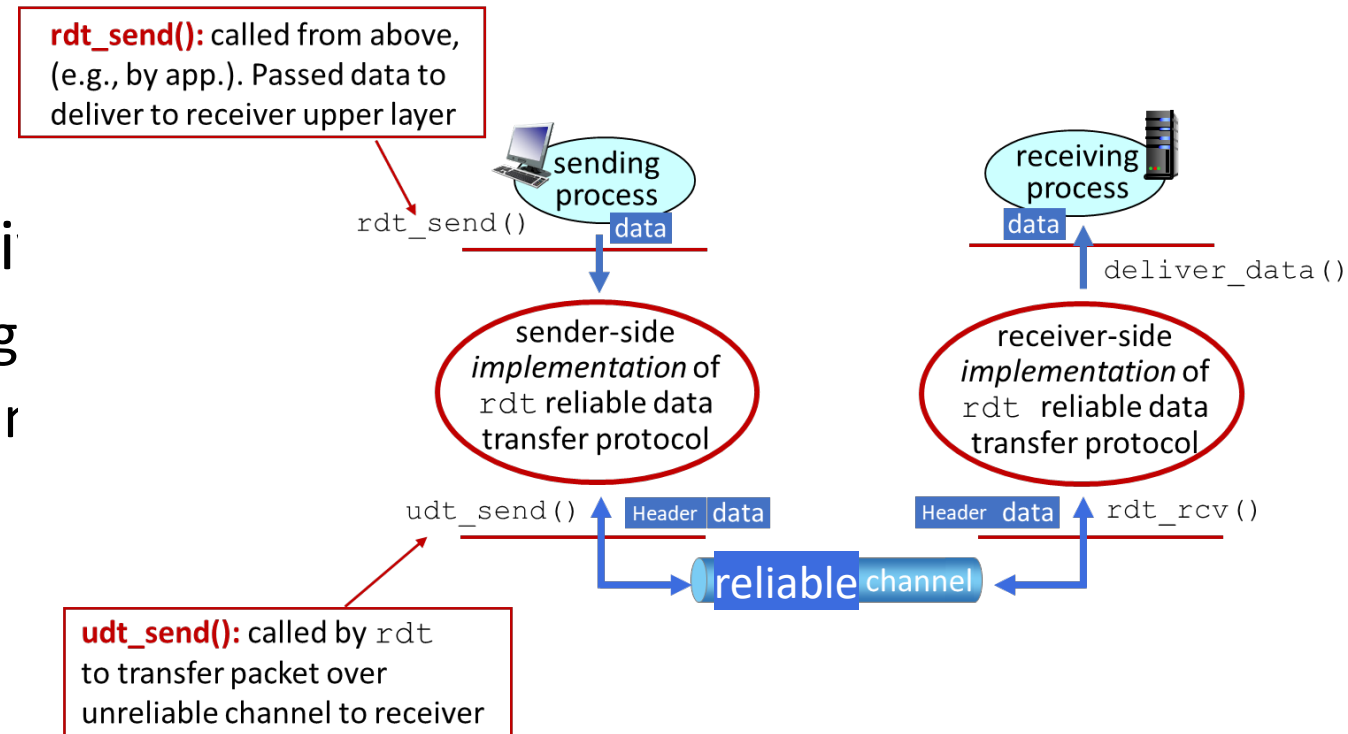
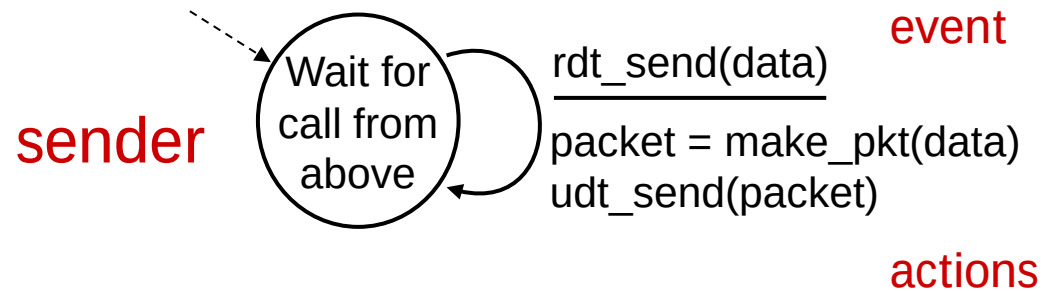
rdt1.0: reliable transfer over a reliable channel

- underlying channel perfectly reliable

- no bit errors
- no loss of packets

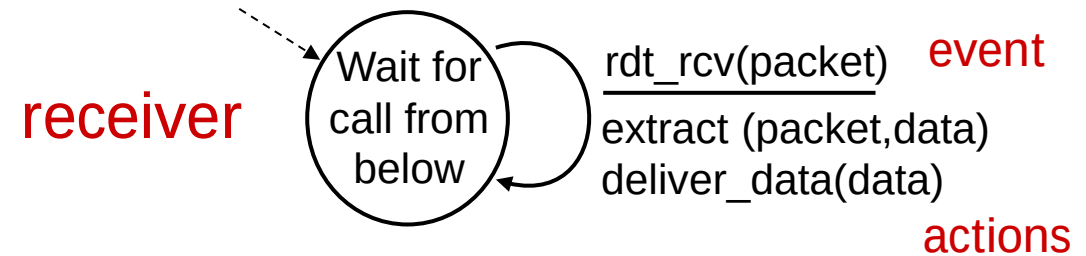
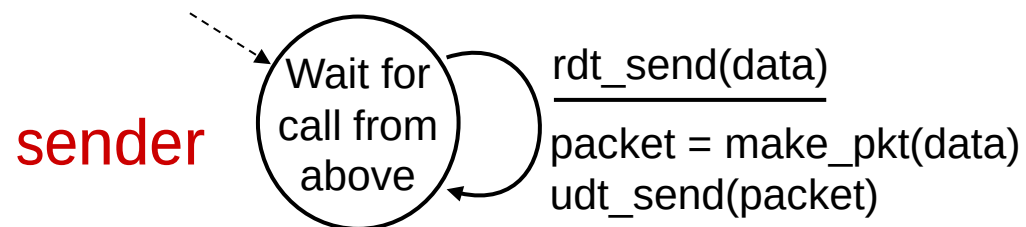
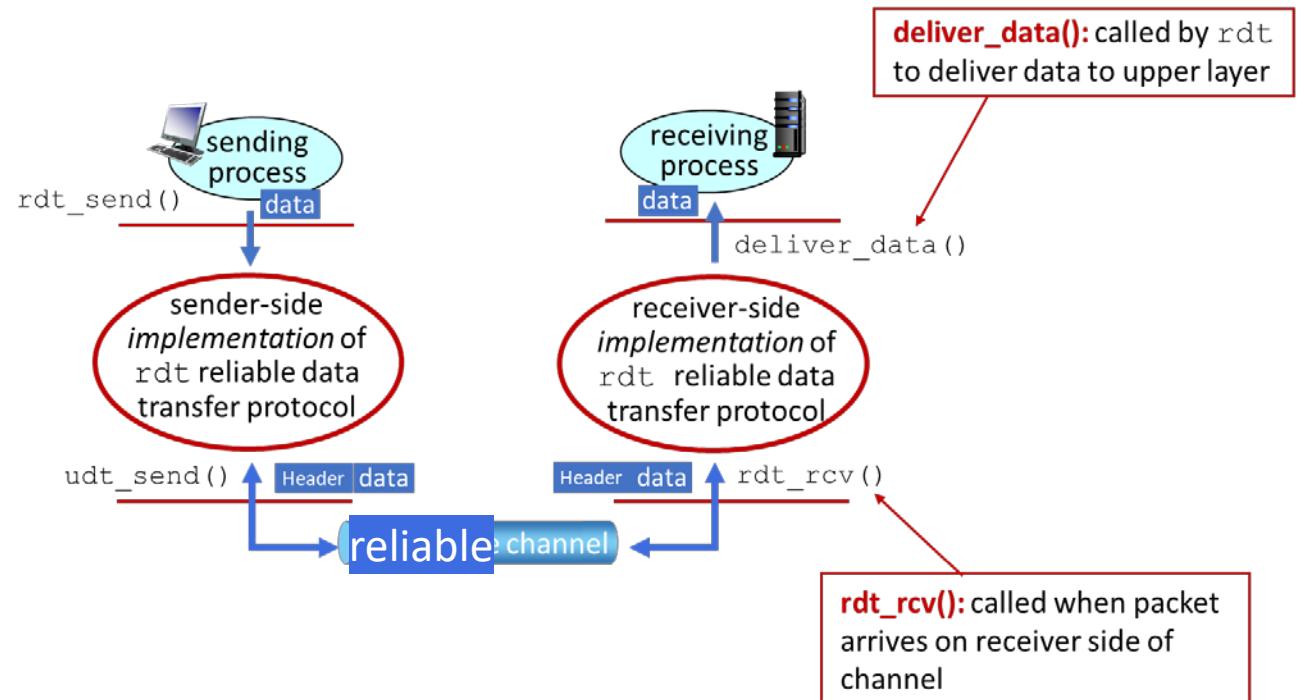
- separate* FSMs for sender, receiver

- sender sends data into underlying
- receiver reads data from underlying



rdt1.0: reliable transfer over a reliable channel

- underlying channel perfectly
 - no bit errors
 - no loss of packets
- *separate* FSMs for sender, receiver
 - sender sends data into underlying channel
 - receiver reads data from underlying channel



rdt2.0: channel with bit errors

- underlying channel may flip bits in packet
 - checksum to detect bit errors
- *the* question: how to recover from errors?

How do humans recover from “errors” during conversation?

rdt2.0: channel with bit errors

- underlying channel may flip bits in packet
 - checksum to detect bit errors
- *the* question: how to recover from errors?

How do humans recover from “errors” during conversation?

- Examples
 - Use of positive acknowledgments
 - Use of negative acknowledgments
 - Retransmission

rdt2.0: channel with bit errors

- underlying channel may flip bits in packet
 - checksum to detect bit errors
- *the* question: how to recover from errors?
 - *acknowledgements (ACKs)*: receiver explicitly tells sender that pkt received OK
 - *negative acknowledgements (NAKs)*: receiver explicitly tells sender that pkt had errors
 - sender *retransmits* pkt on receipt of NAK

rdt2.0: channel with bit errors

- underlying channel may flip bits in packet
 - checksum to detect bit errors
- *the* question: how to recover from errors?
 - *acknowledgements (ACKs)*: receiver explicitly tells sender that pkt received OK
 - *negative acknowledgements (NAKs)*: receiver explicitly tells sender that pkt had errors
 - sender *retransmits* pkt on receipt of NAK

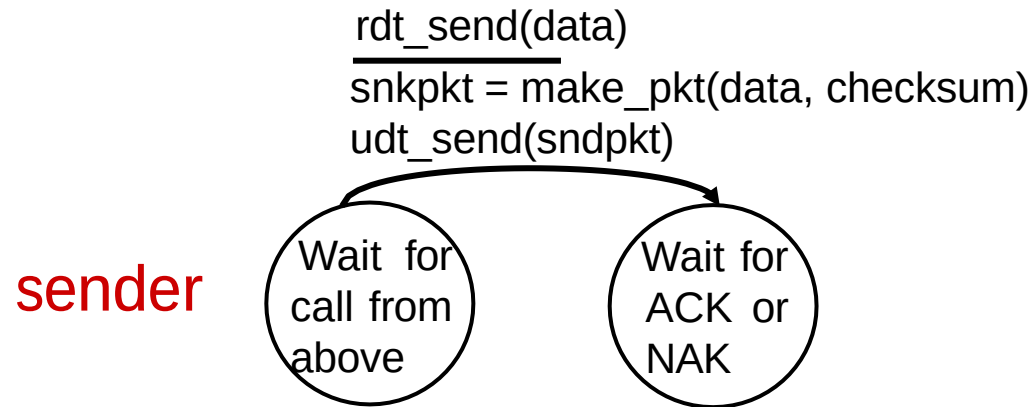
stop and wait

sender sends one packet, then waits for receiver's response

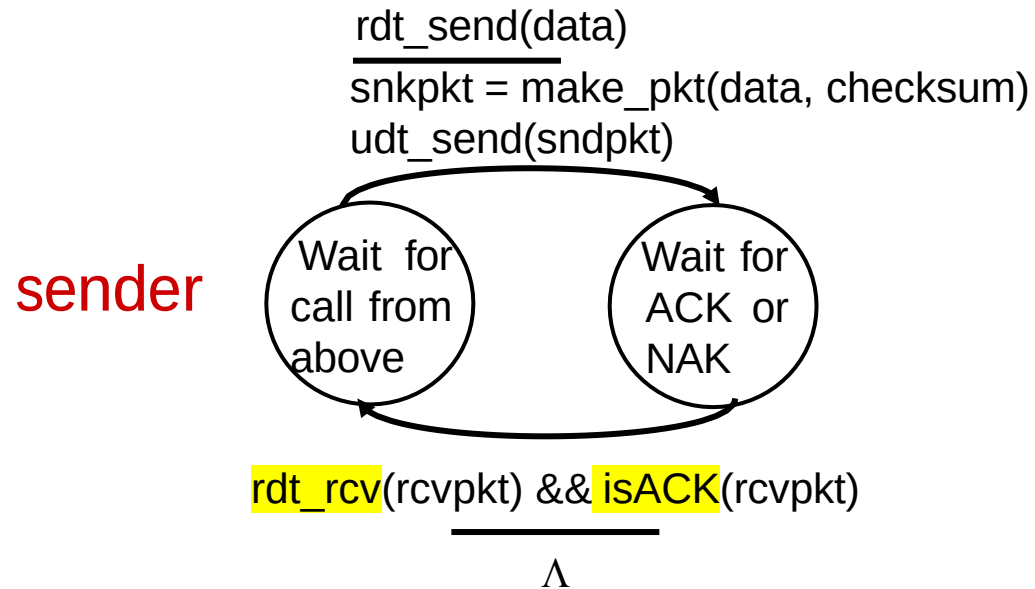
rdt2.0: FSM specifications



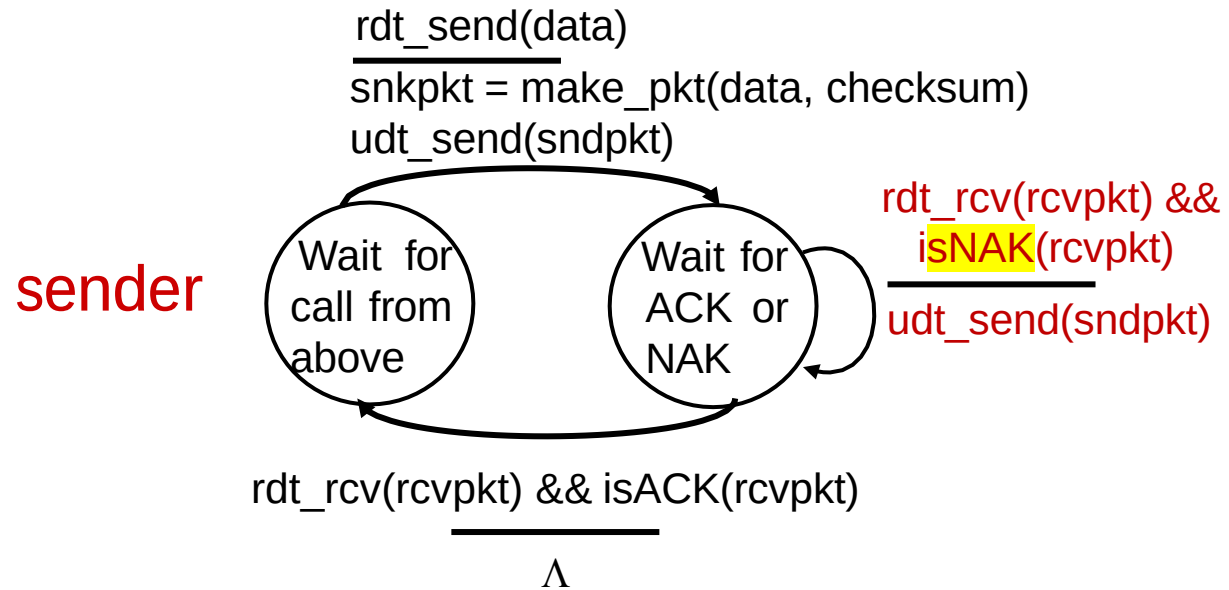
rdt2.0: FSM specifications



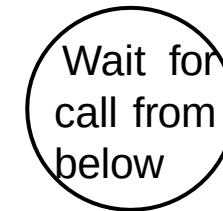
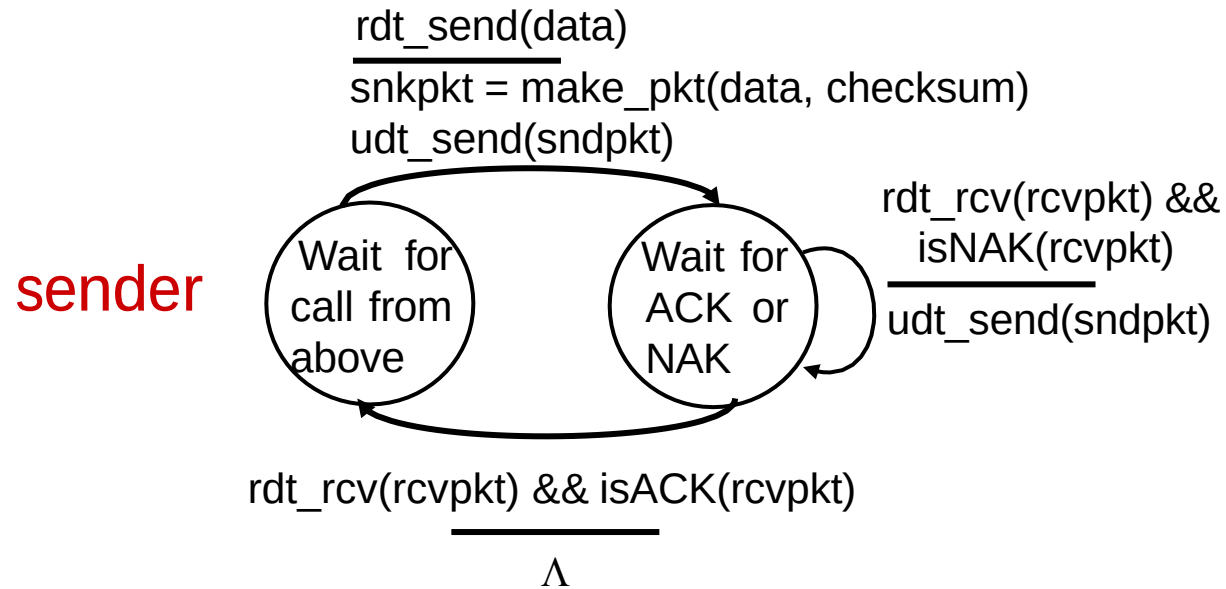
rdt2.0: FSM specifications



rdt2.0: FSM specifications

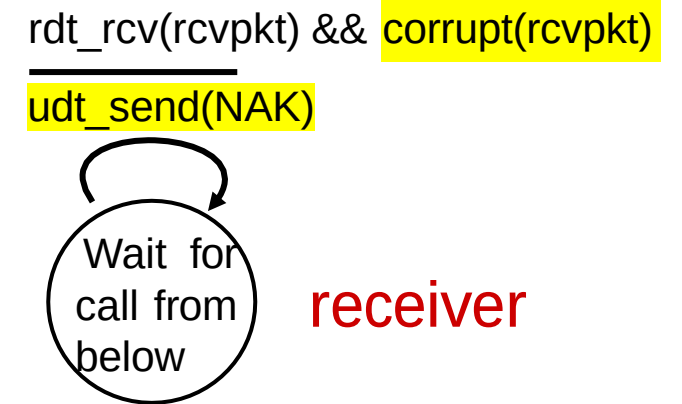
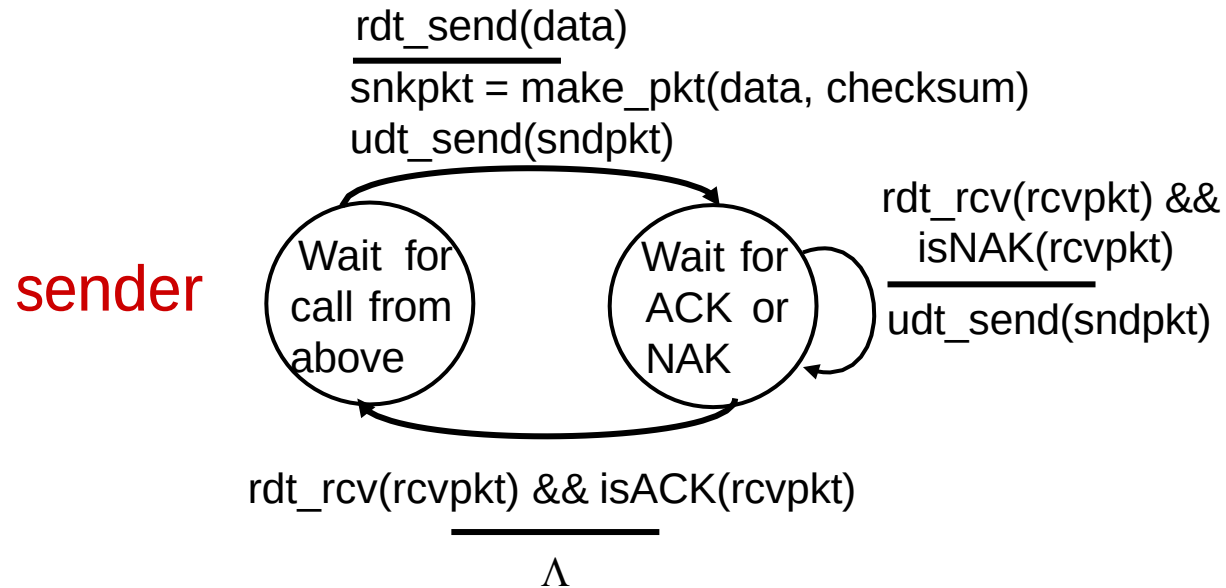


rdt2.0: FSM specifications

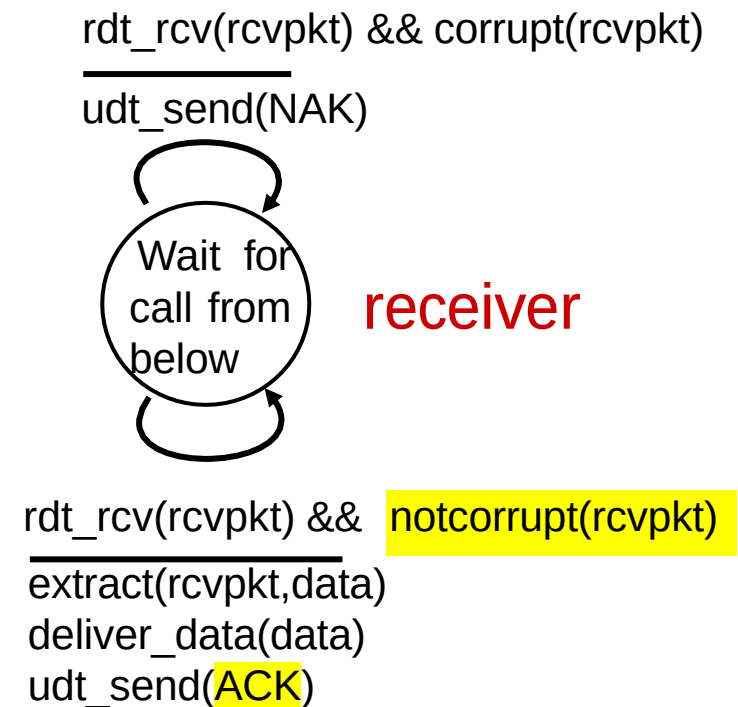
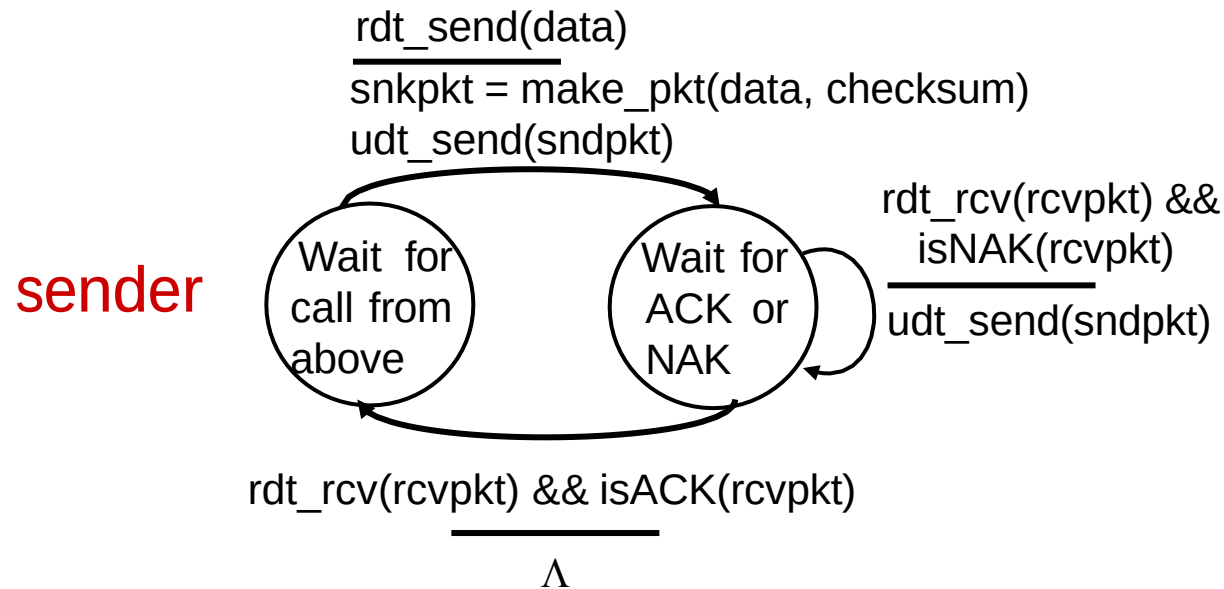


receiver

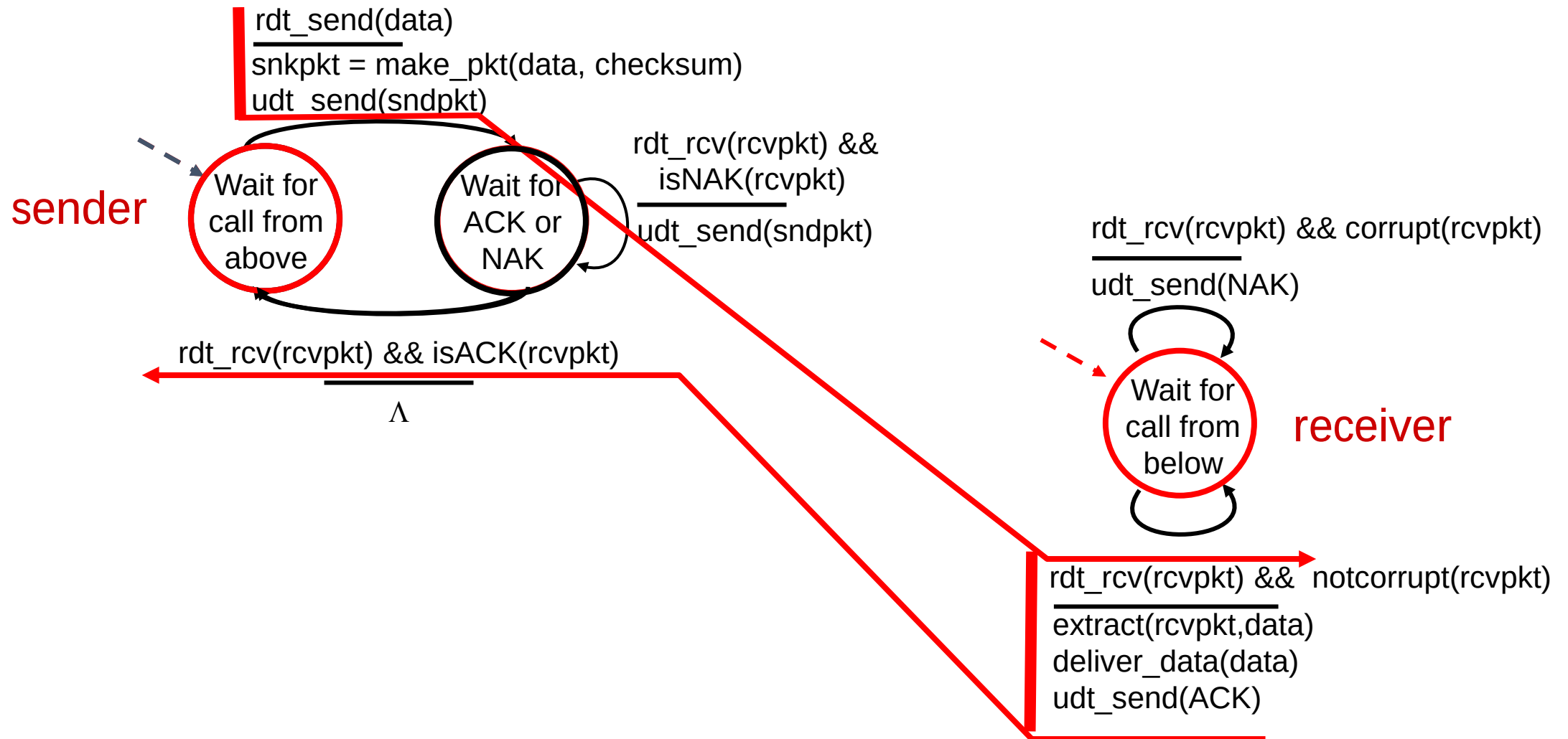
rdt2.0: FSM specifications



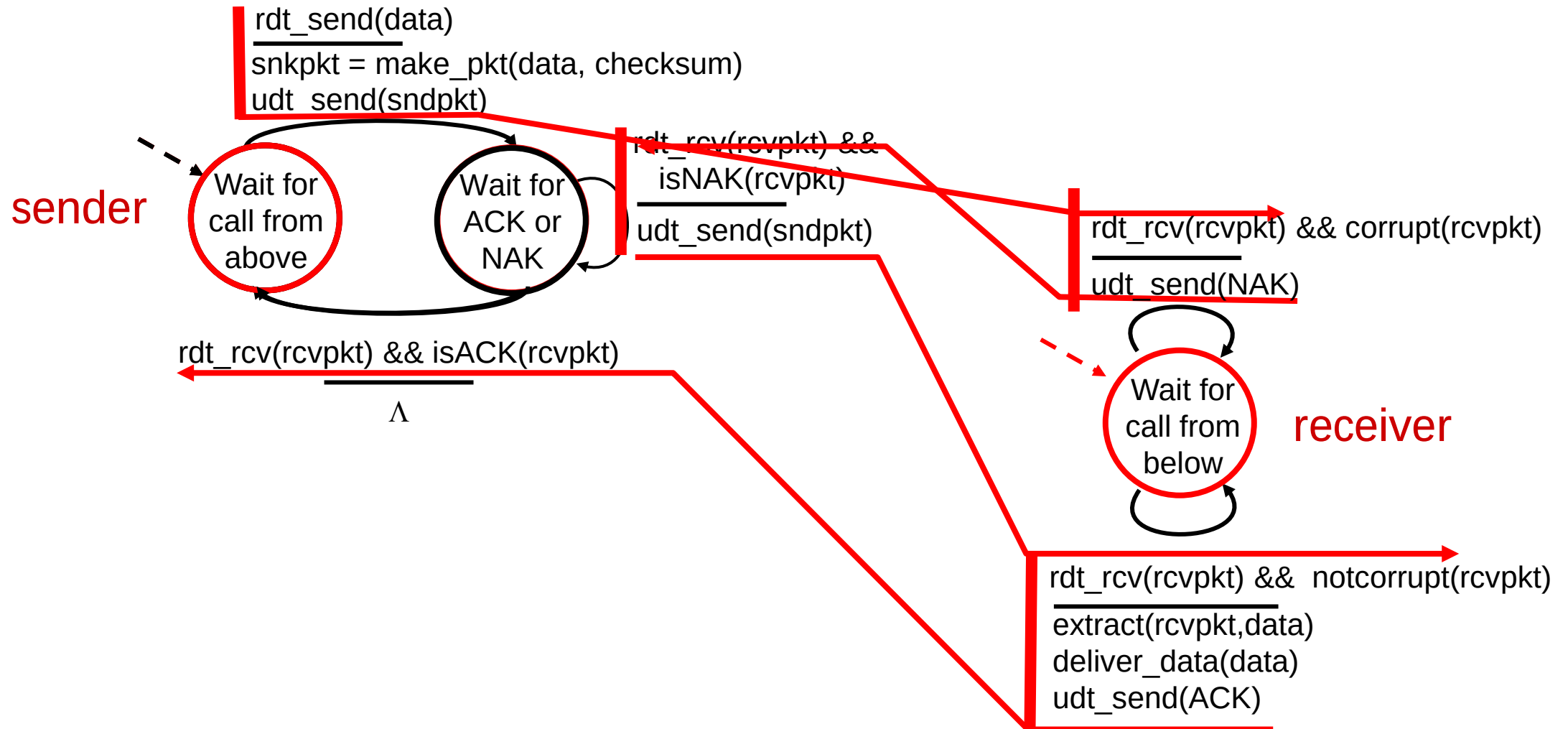
rdt2.0: FSM specifications



rdt2.0: operation with no errors



rdt2.0: corrupted packet scenario



rdt2.0 has a fatal flaw!

what happens if ACK/NAK
corrupted?

- sender doesn't know what happened at receiver!
- Retransmit? cause duplicate

rdt2.0 has a fatal flaw!

what happens if ACK/NAK corrupted?

- sender doesn't know what happened at receiver!
- Retransmit? cause duplicate

handling duplicates:

- sender retransmits current pkt if ACK/NAK corrupted
- sender adds *sequence number* to each pkt
- receiver discards (doesn't deliver up) duplicate pkt

rdt2.1

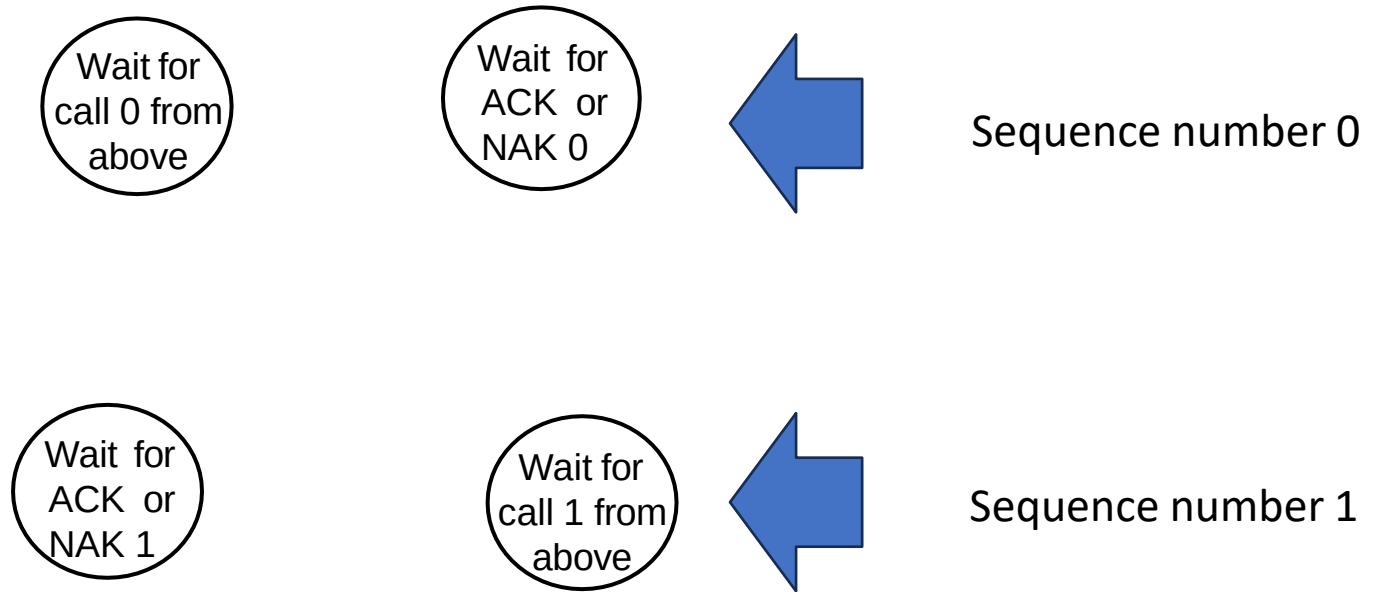
— stop and wait —

sender sends one packet, then waits for receiver's response

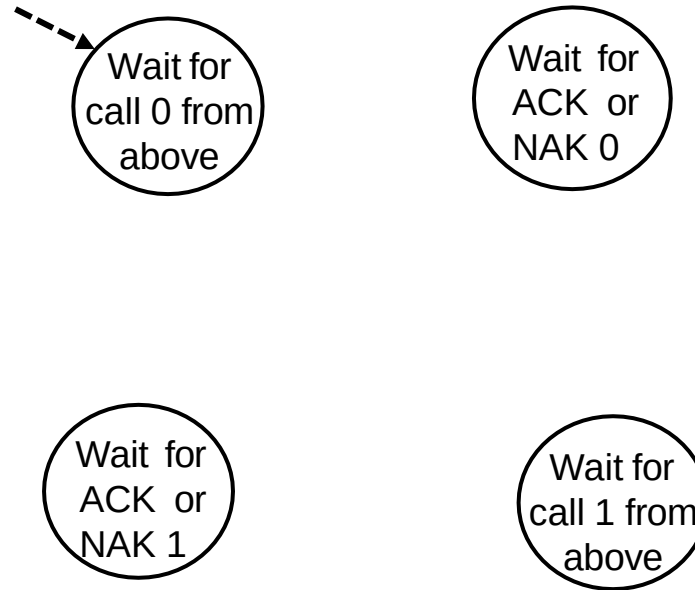


Sequence number

rdt2.1: sender, handling garbled ACK/NAKs

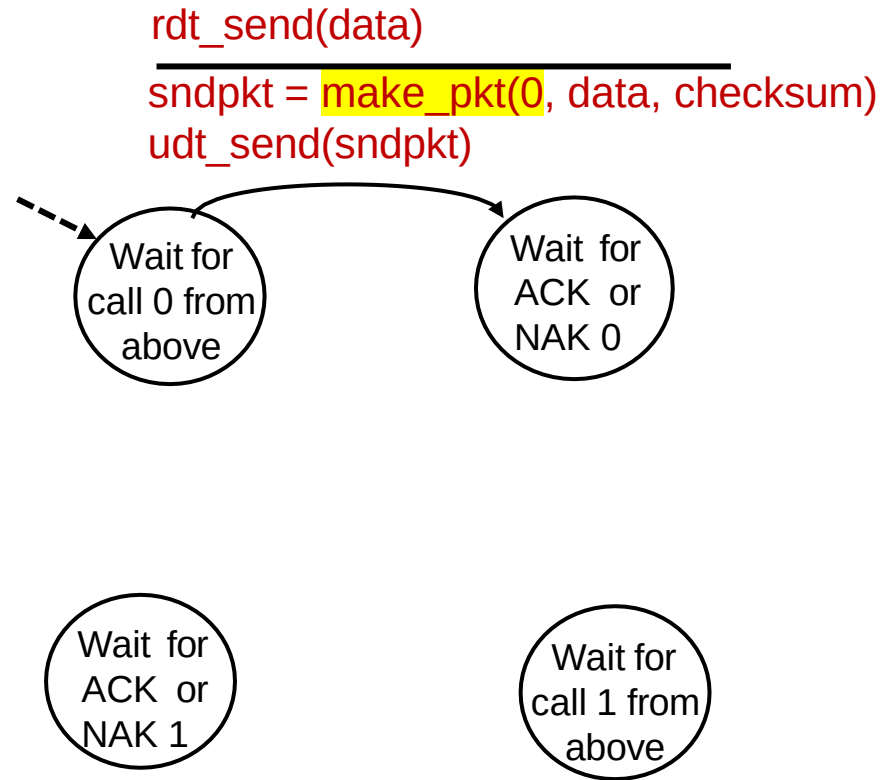


rdt2.1: sender, handling garbled ACK/NAKs

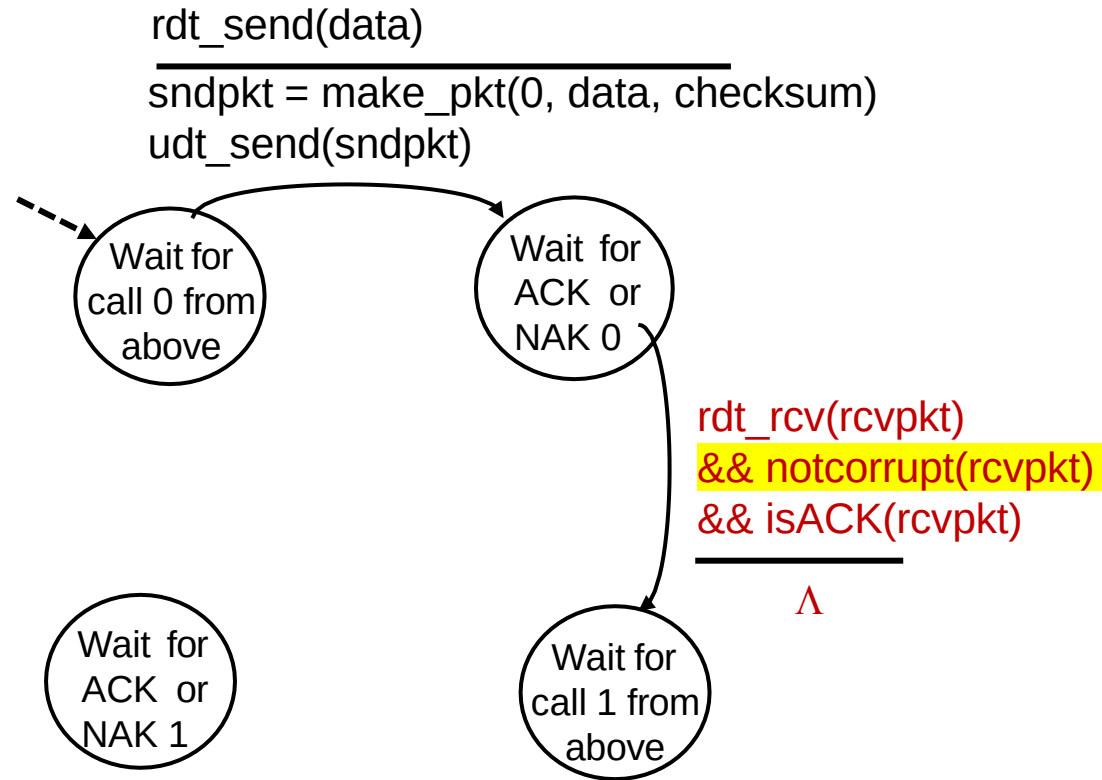


Look clockwise around the states
First case: no corruptions

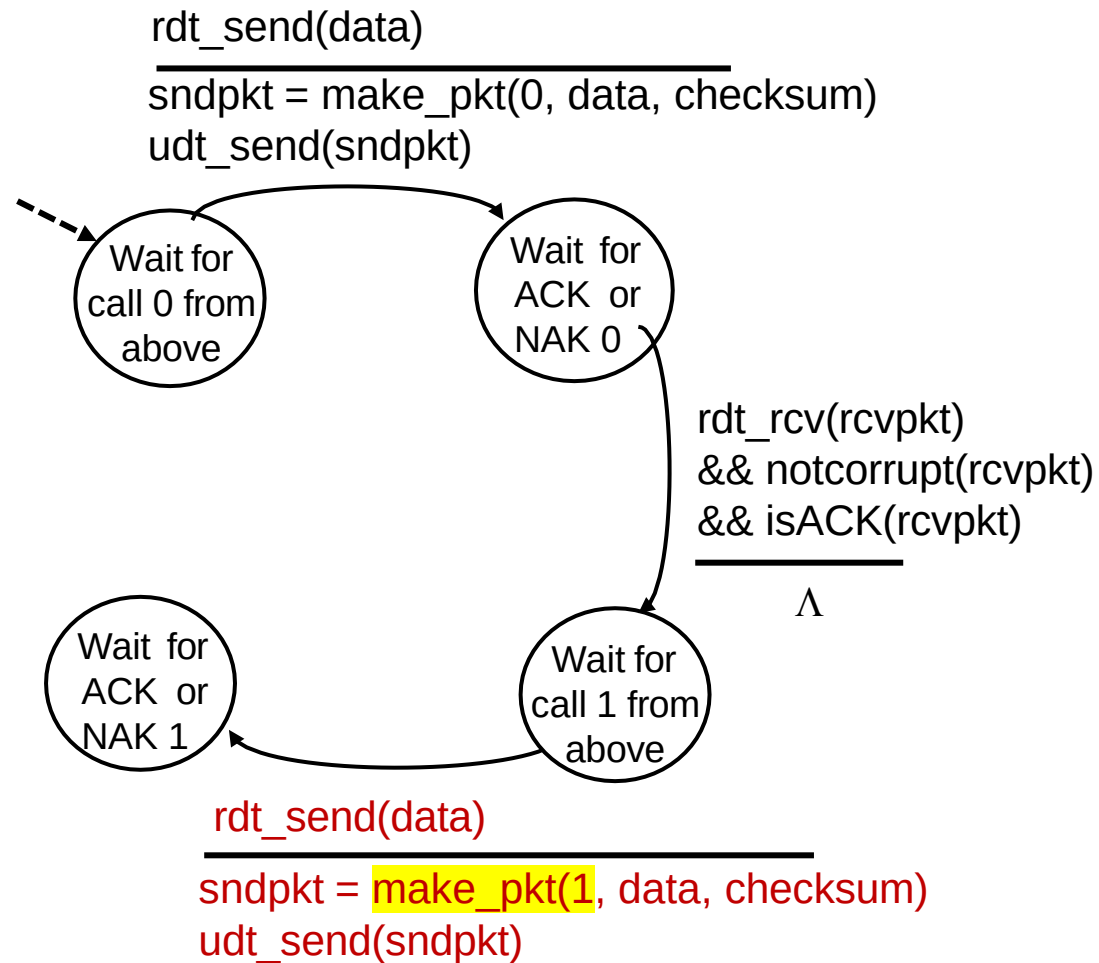
rdt2.1: sender, handling garbled ACK/NAKs



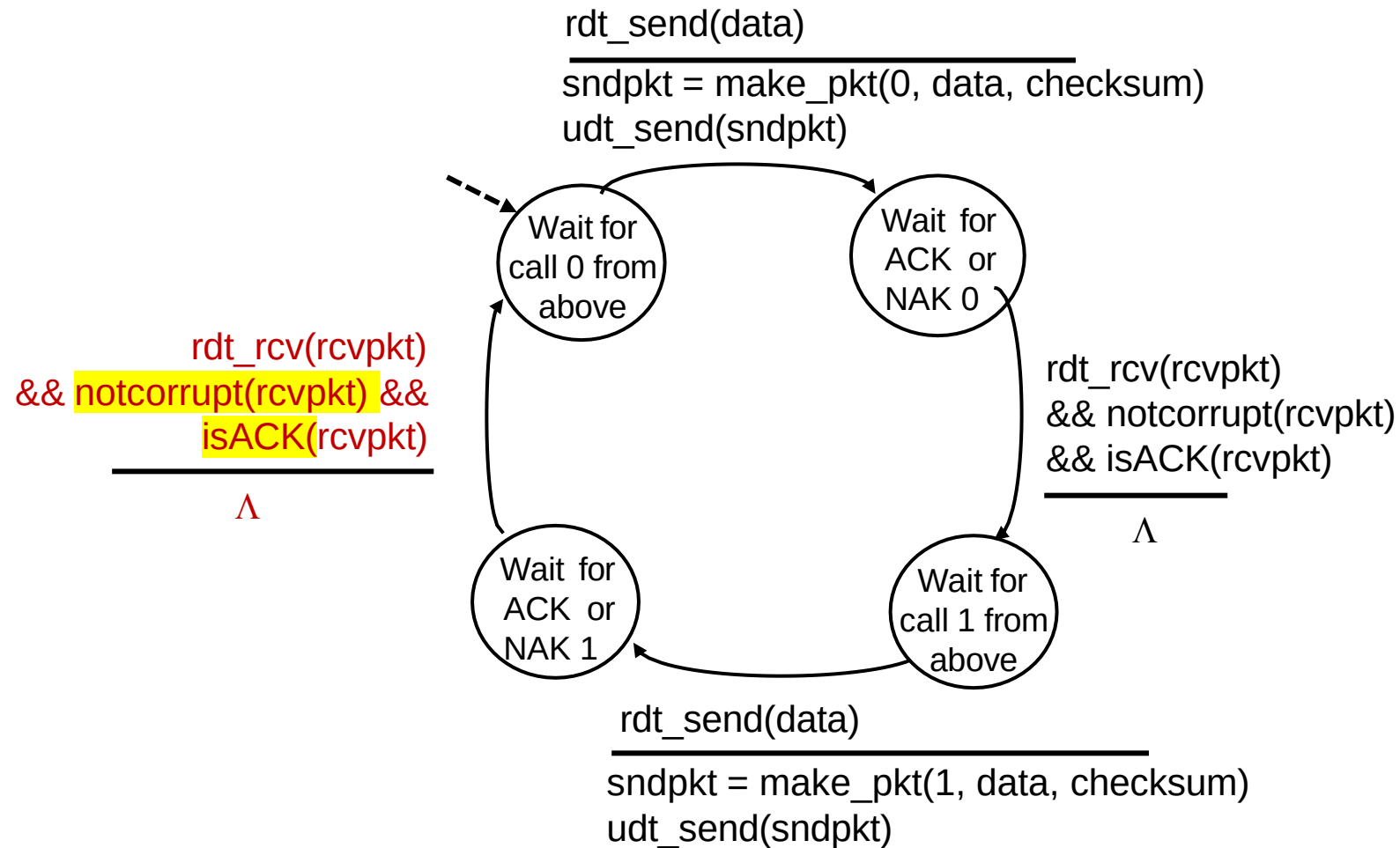
rdt2.1: sender, handling garbled ACK/NAKs



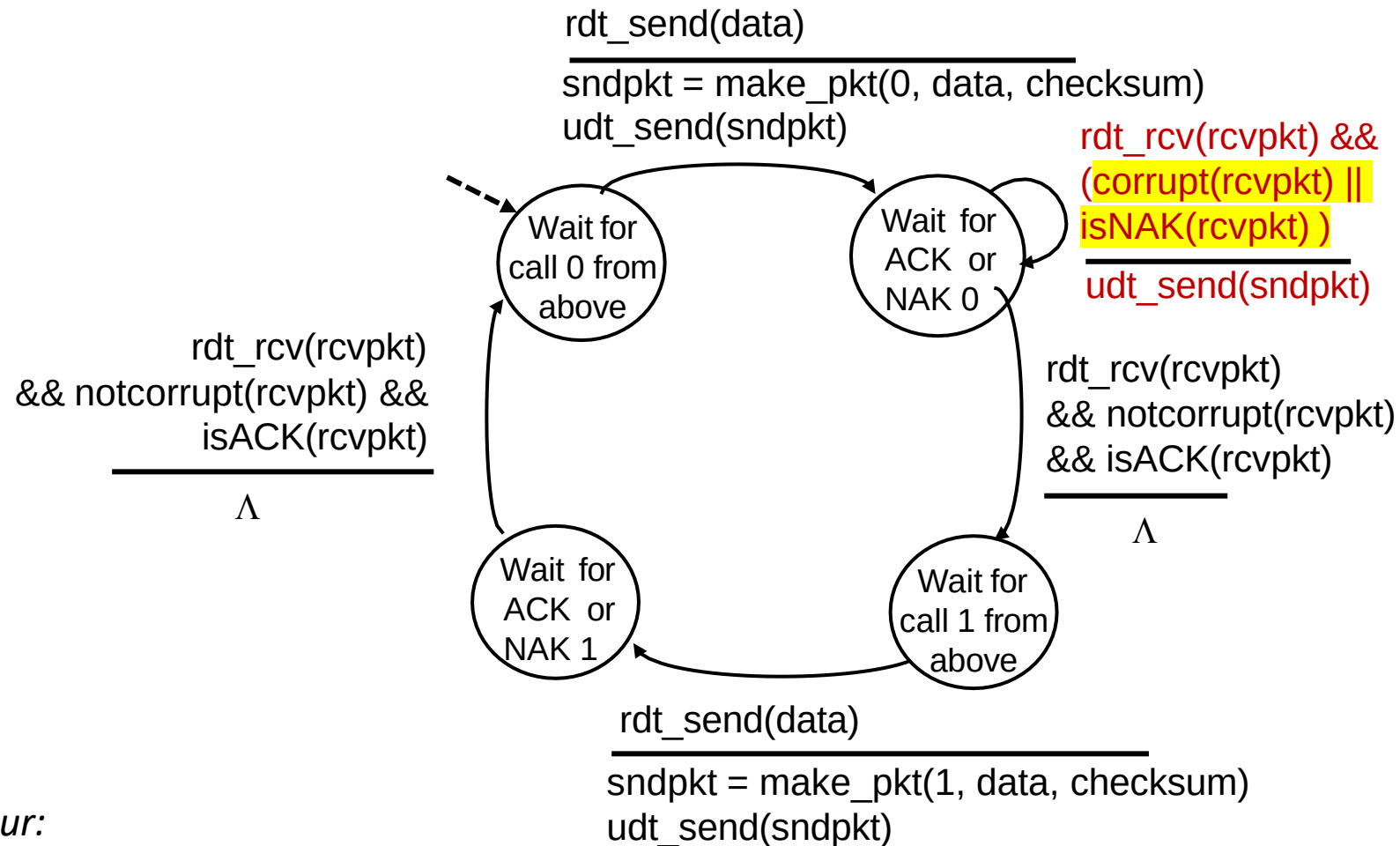
rdt2.1: sender, handling garbled ACK/NAKs



rdt2.1: sender, handling garbled ACK/NAKs



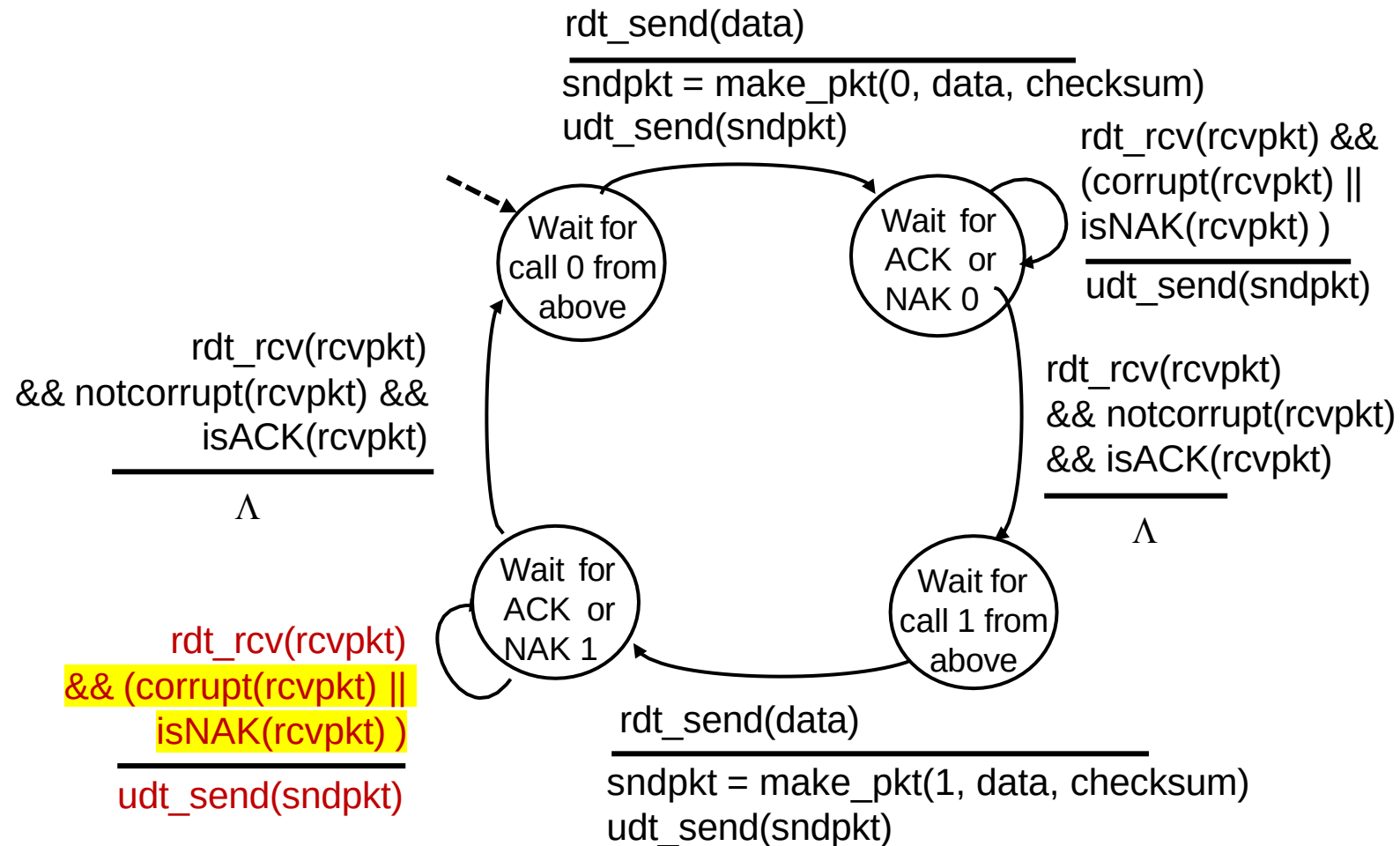
rdt2.1: sender, handling garbled ACK/NAKs



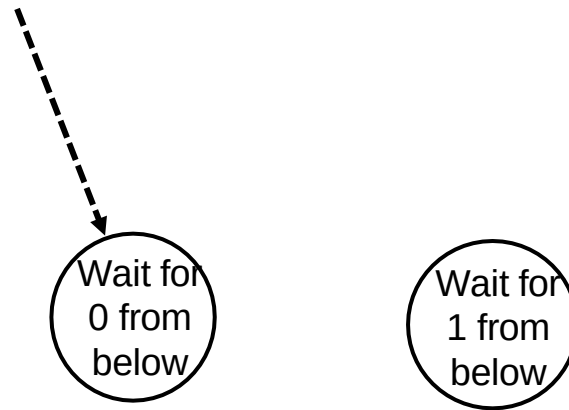
Bit errors can occur:

- In a corrupted packet
- In the ACK or NAK

rdt2.1: sender, handling garbled ACK/NAKs

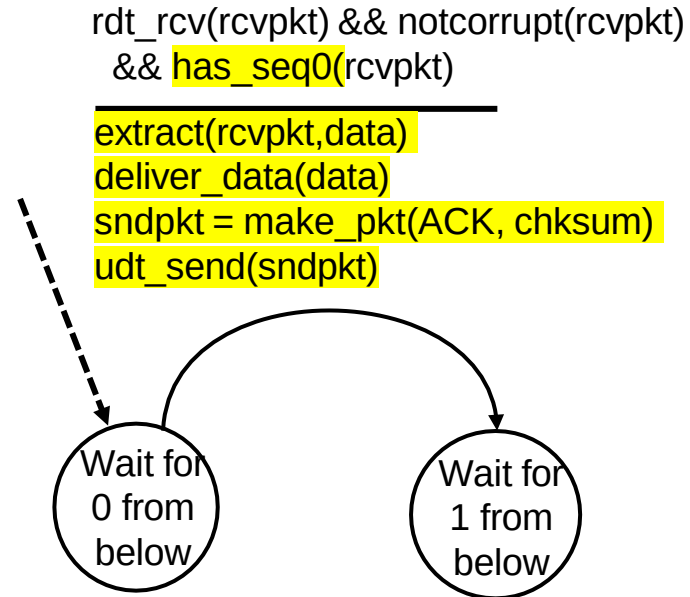


rdt2.1: receiver, handling garbled ACK/NAKs

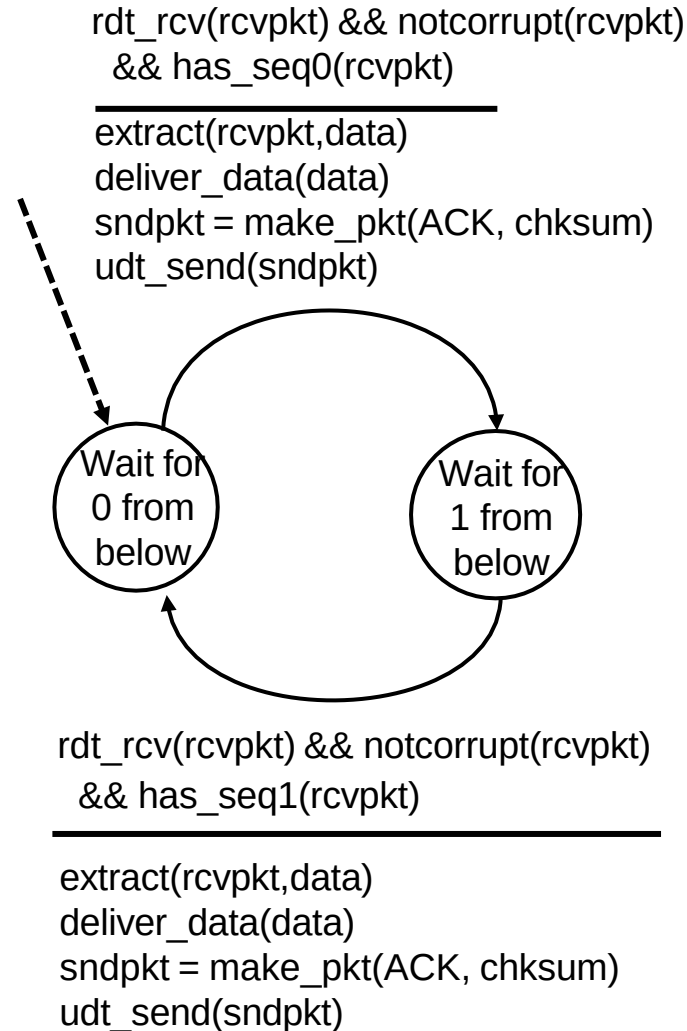


rdt2.1: receiver, handling garbled ACK/NAKs

A non-corrupted,
sequence number 0
pack is received



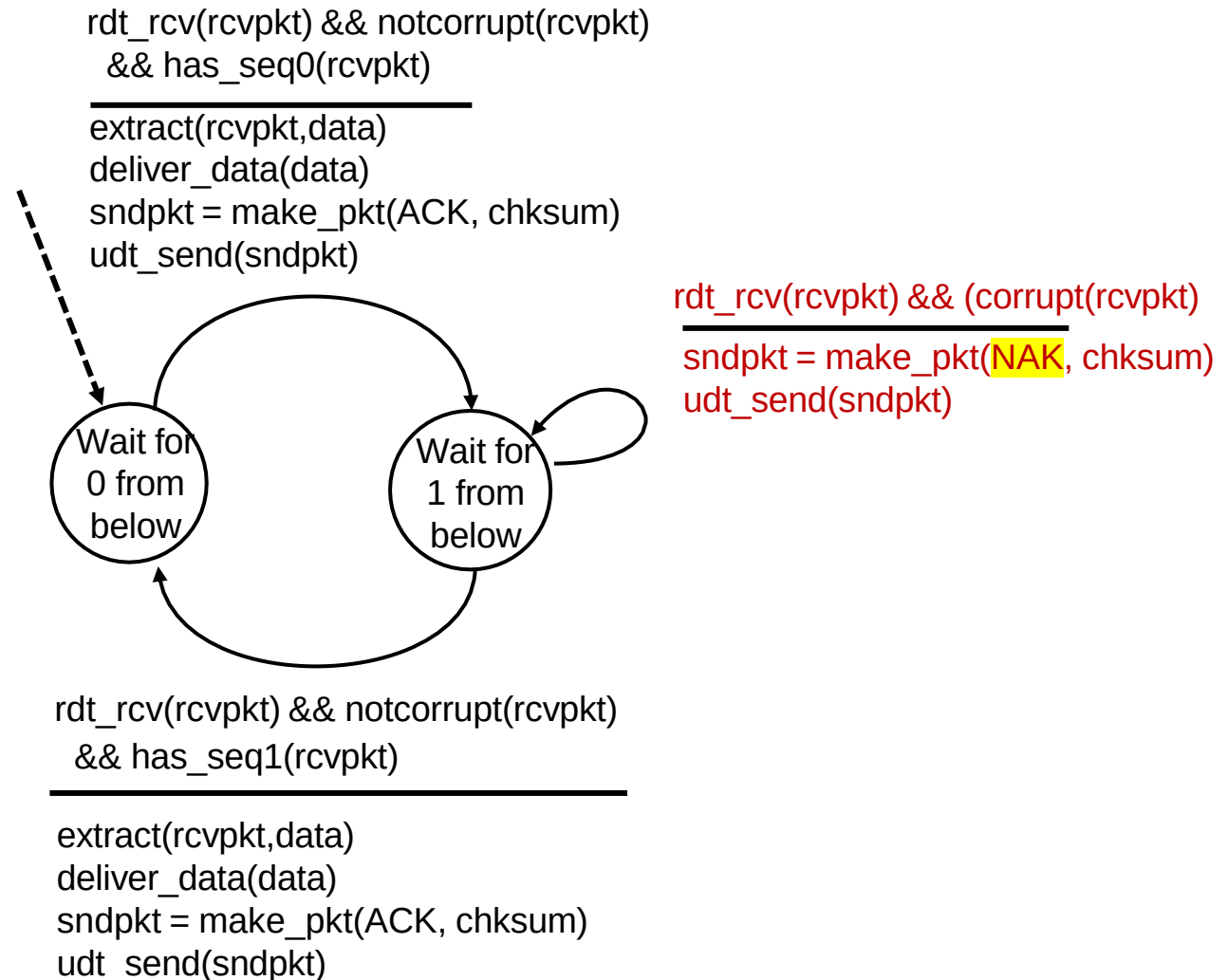
rdt2.1: receiver, handling garbled ACK/NAKs



A non-corrupted,
sequence number 1
pack is received

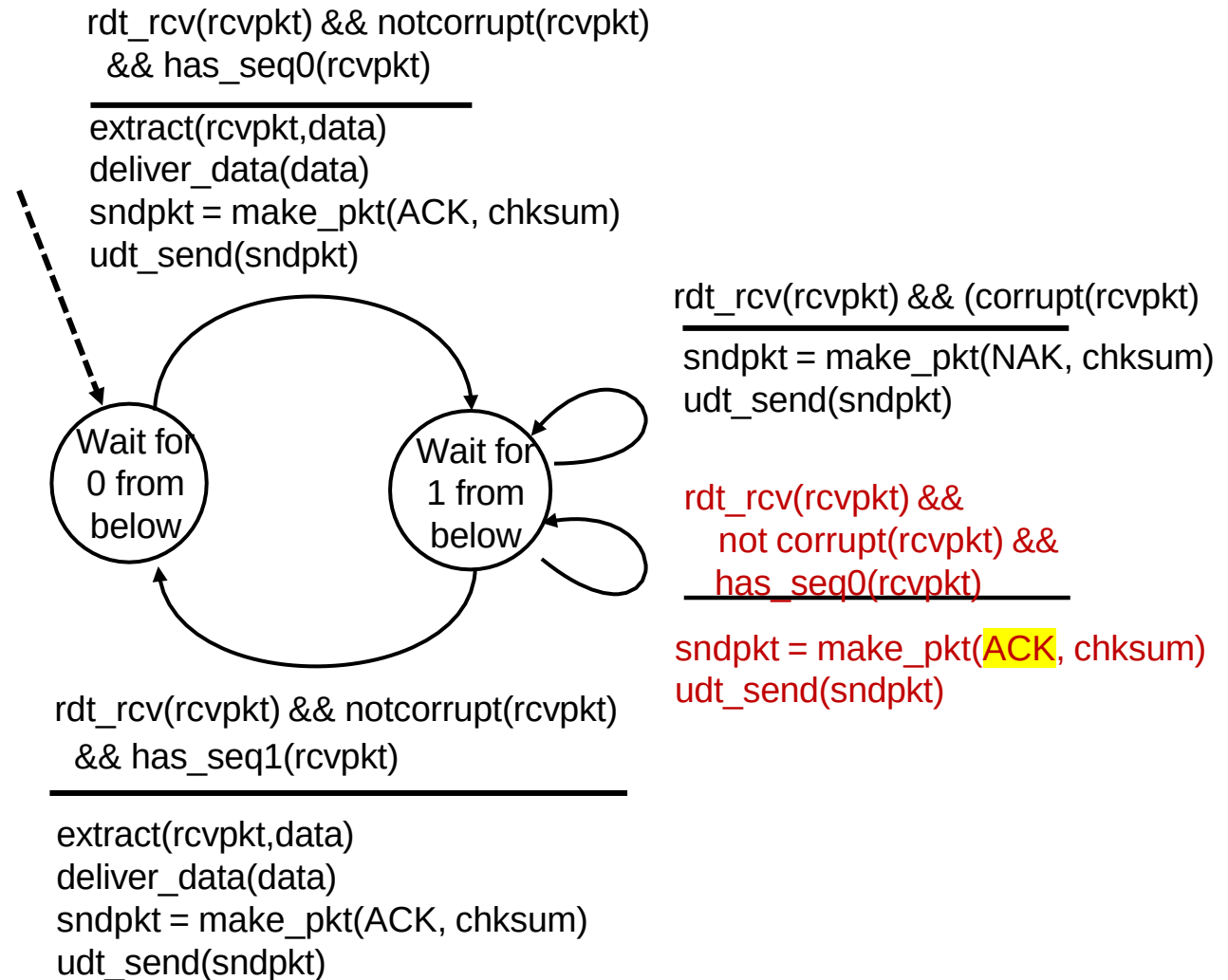
rdt2.1: receiver, handling garbled ACK/NAKs

A corrupted, pack is received

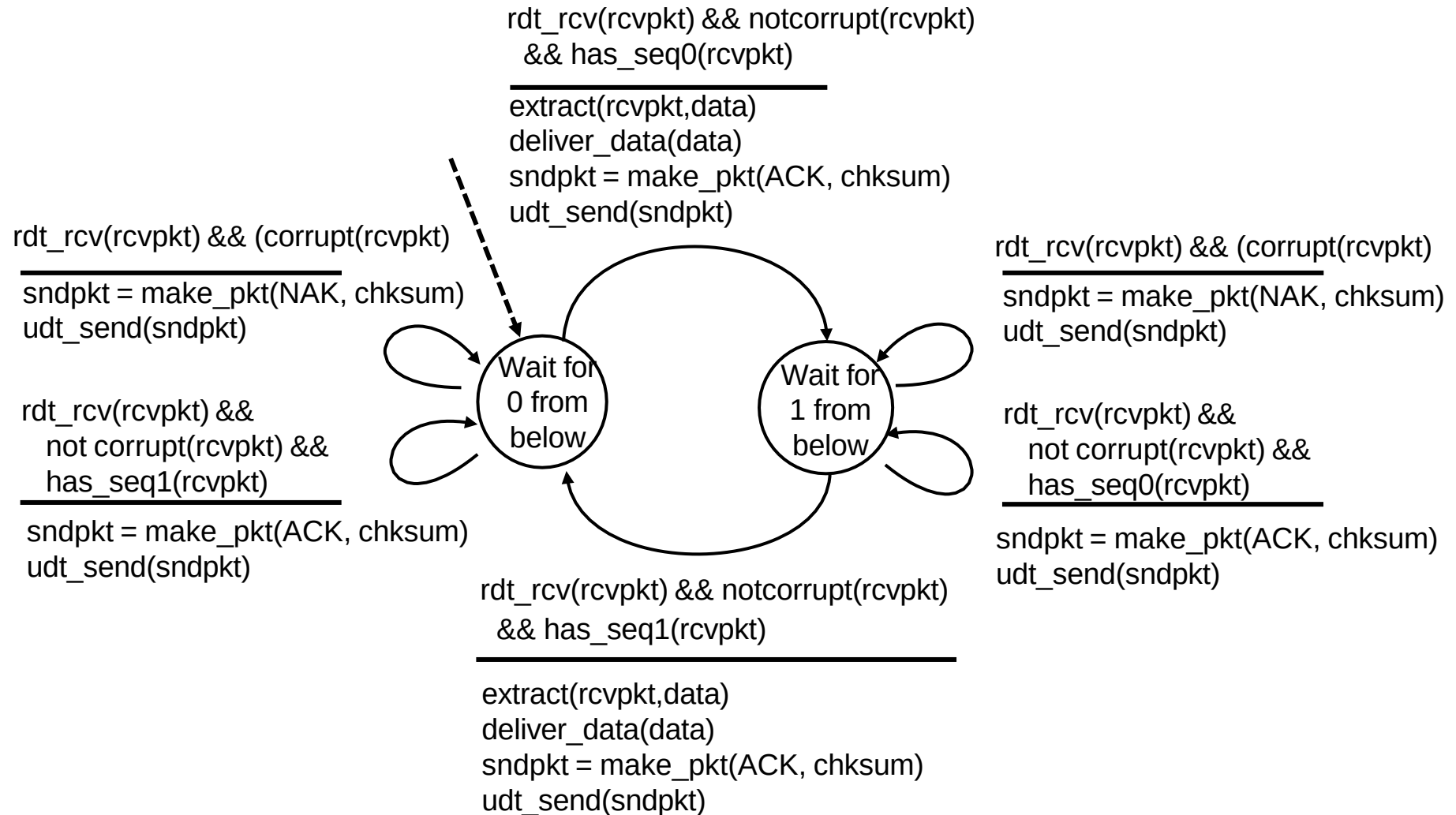


rdt2.1: receiver, handling garbled ACK/NAKs

A sequence number 0 packet is received, while waiting for a sequence number 1 packet



rdt2.1: receiver, handling garbled ACK/NAKs

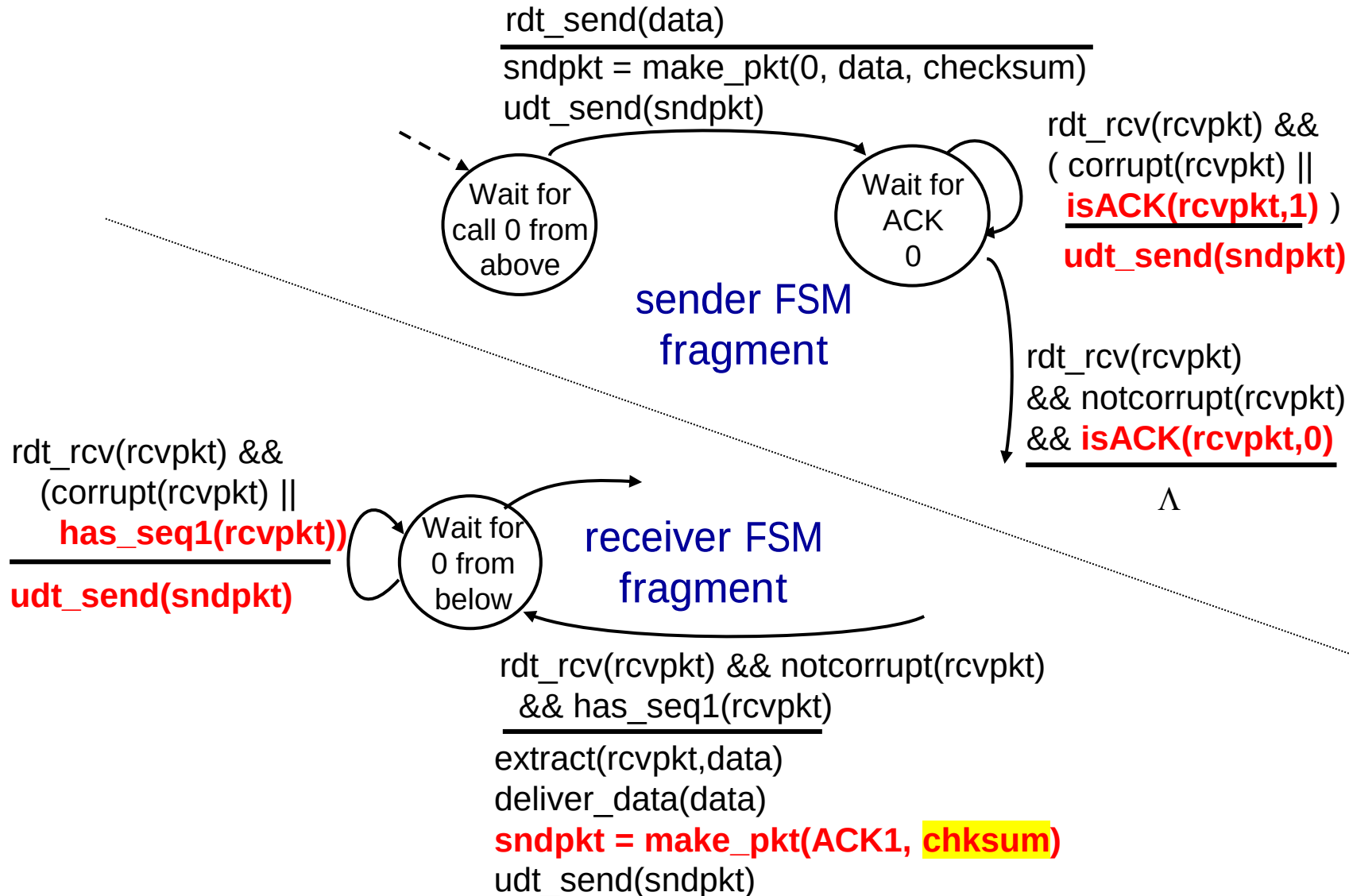


rdt2.2: a NAK-free protocol

- same functionality as rdt2.1, using ACKs only
- instead of NAK, receiver sends ACK for last pkt received OK
 - receiver must *explicitly* include seq # of pkt being ACKed
- duplicate ACK at sender results in same action as NAK:
retransmit current pkt

As we will see, TCP uses this approach to be NAK-free

rdt2.2: sender, receiver fragments



E.g.,
packet 0 – correctly received
packet 1 – corrupted

The receiver: resends an ACK for packet 0 again

The sender: receives the same ACK for packet 0, and resend packet 1

rdt protocol mechanisms:

- error detection (e.g., checksum)
- ACKs, NAKs
- retransmission
- sequence numbers (duplicate detection)

What happens if packets are actually **lost** in the communication channel?

rdt3.0: channels with errors *and* loss

New channel assumption: underlying channel can also *lose* packets (data, ACKs)

- checksum, sequence #s, ACKs, retransmissions will be of help ... but not quite enough

Q: How do *humans* handle lost sender-to-receiver words in conversation?

rdt3.0: channels with errors *and* loss

Approach: sender waits “reasonable” amount of time for ACK



timeout

rdt3.0: channels with errors *and* loss

Approach: sender waits “reasonable” amount of time for ACK

- retransmits if no ACK received in this time
- if pkt (or ACK) just delayed (not lost):
 - retransmission will be duplicate, but seq #s already handles this!
 - receiver must specify seq # of packet being ACKed



timeout

rdt3.0: channels with errors *and* loss

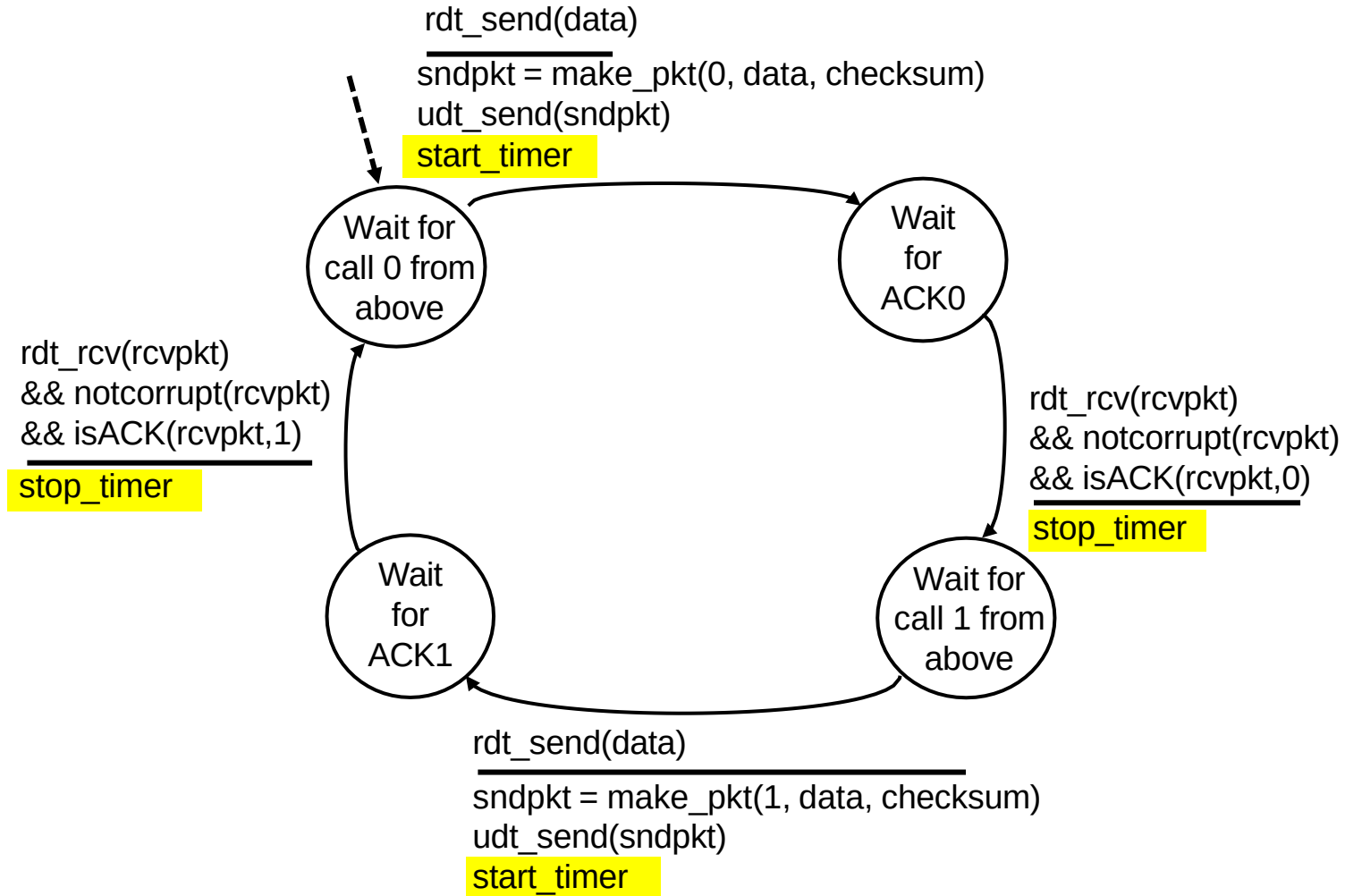
Approach: sender waits “reasonable” amount of time for ACK

- retransmits if no ACK received in this time
- if pkt (or ACK) just delayed (not lost):
 - retransmission will be duplicate, but seq #s already handles this!
 - receiver must specify seq # of packet being ACKed
- use countdown timer to interrupt after “reasonable” amount of time

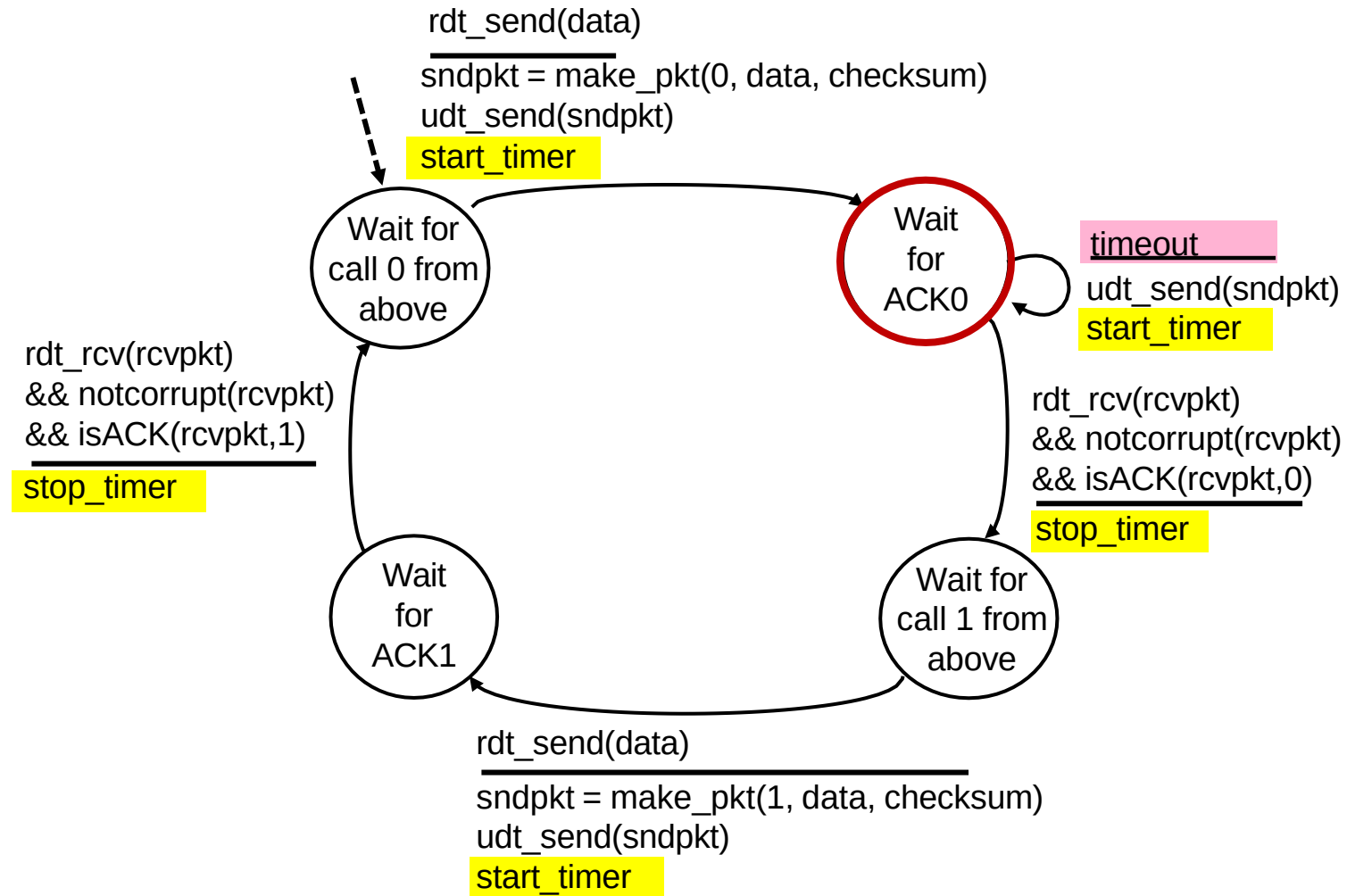


timeout

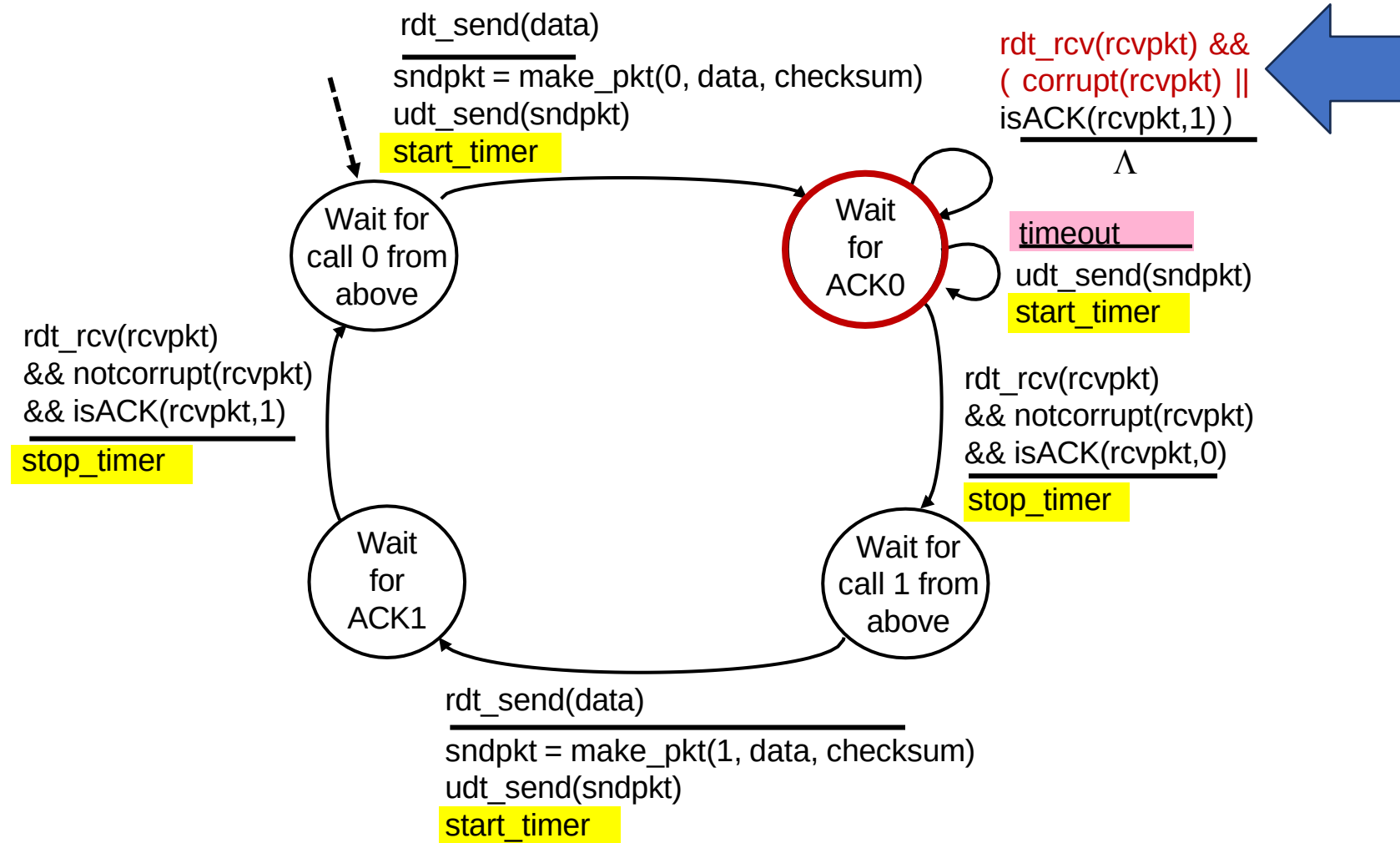
rdt3.0 sender



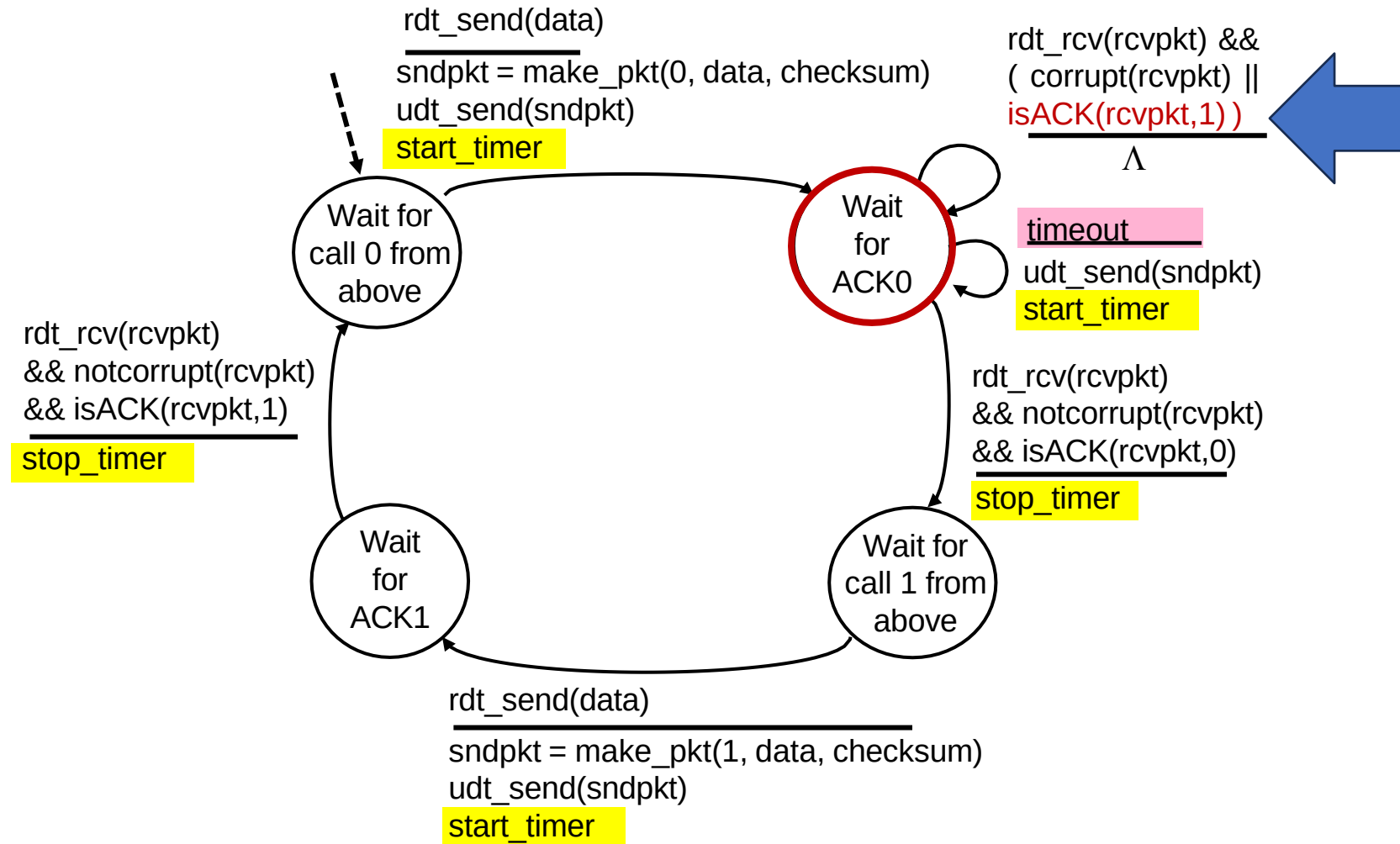
rdt3.0 sender



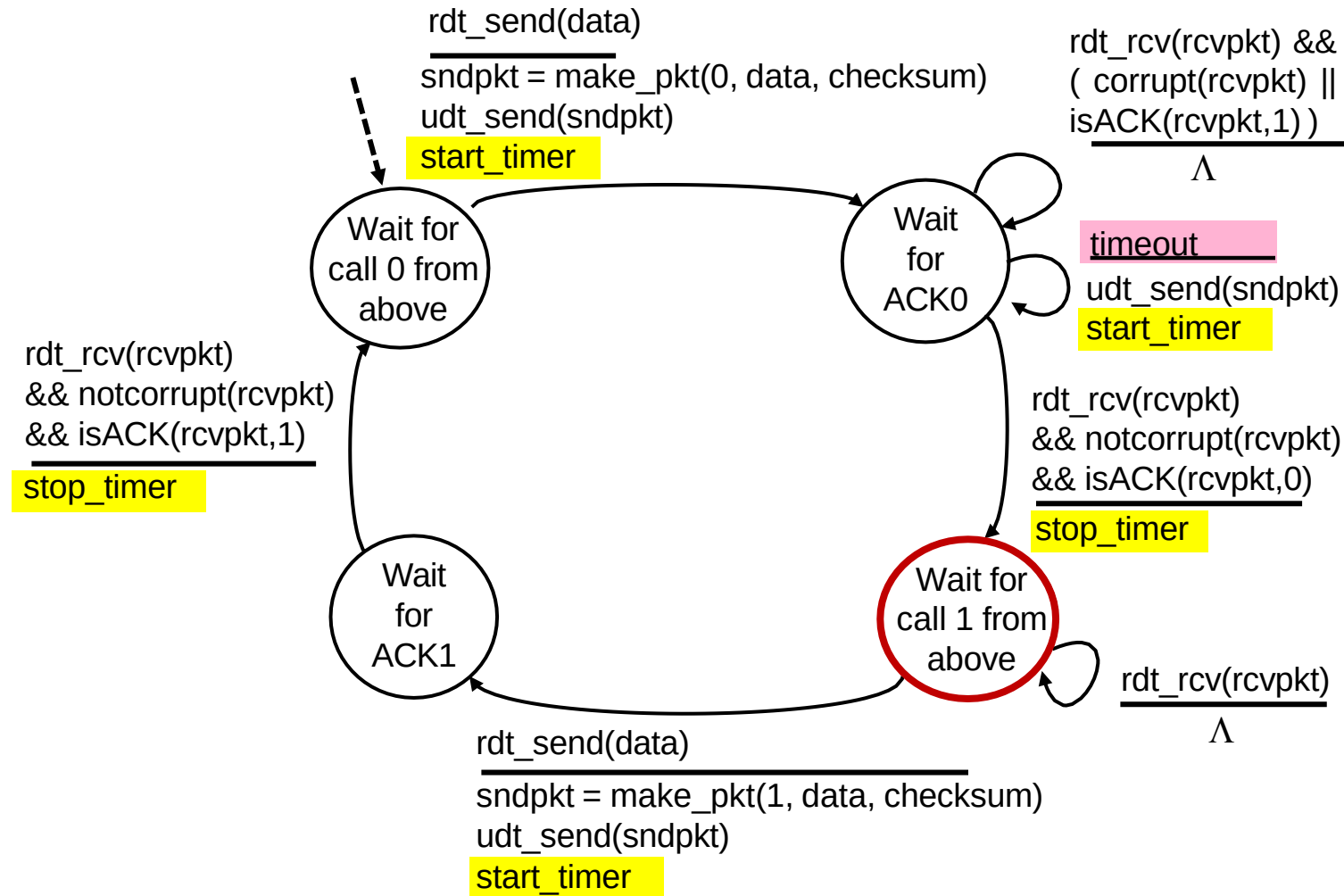
rdt3.0 sender



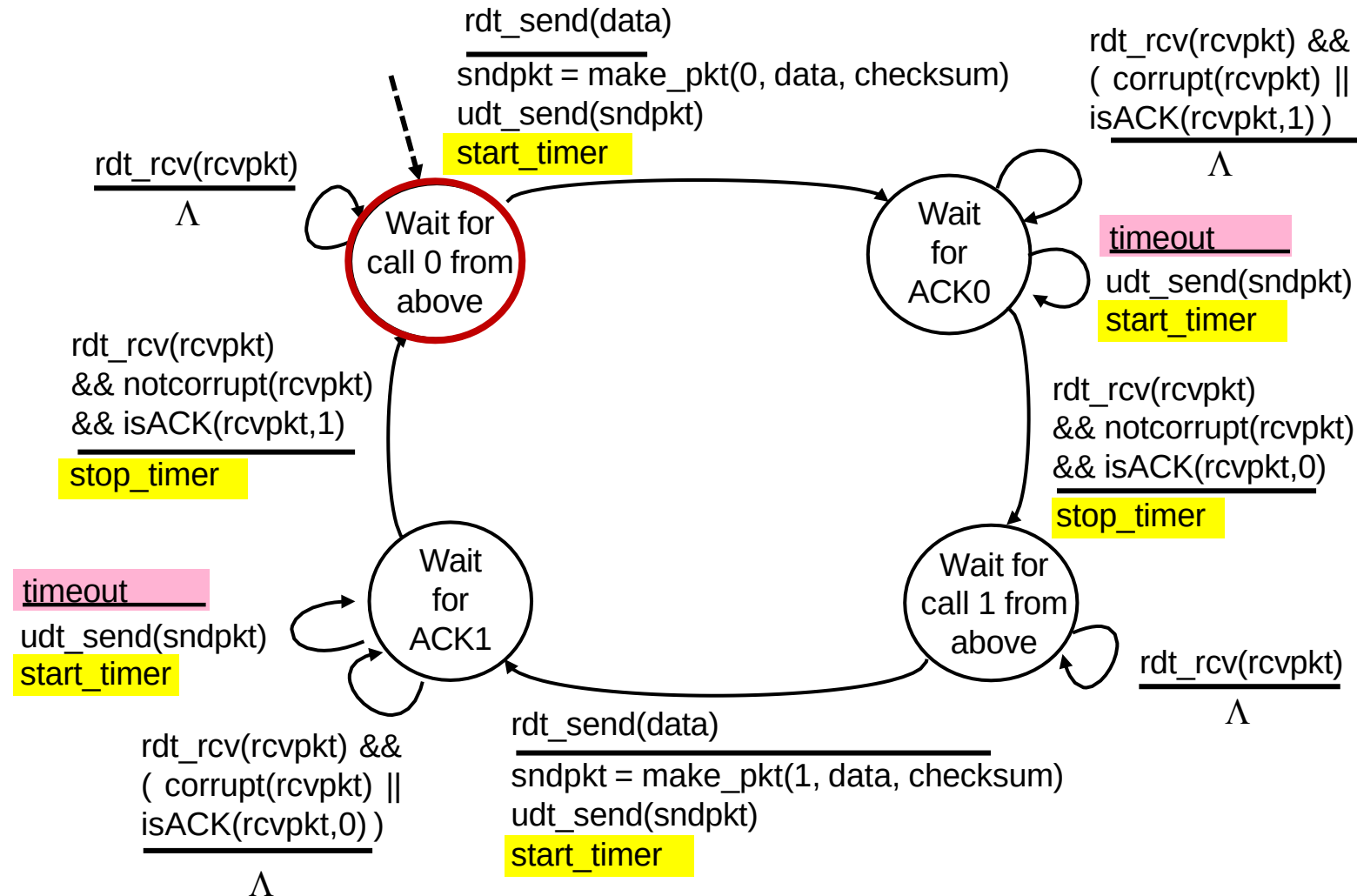
rdt3.0 sender



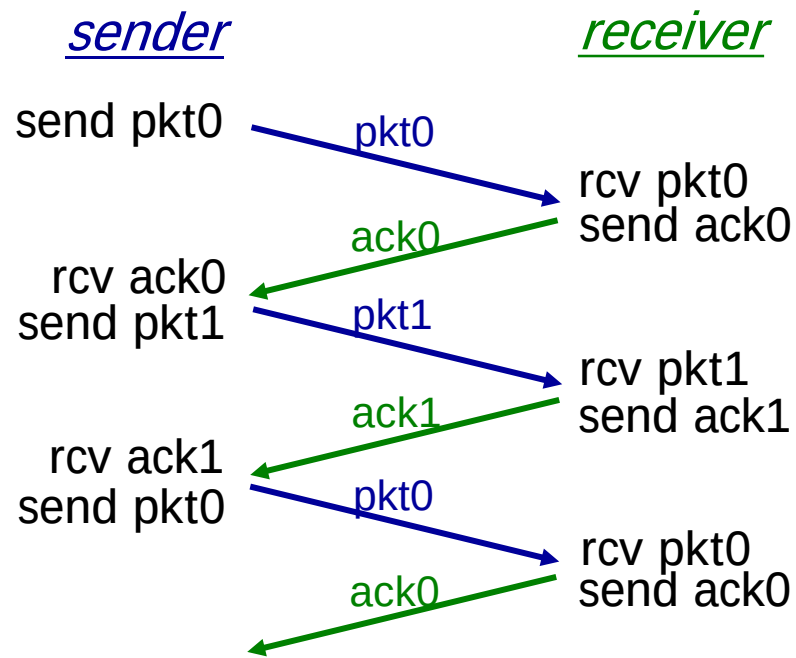
rdt3.0 sender



rdt3.0 sender



rdt3.0 in action

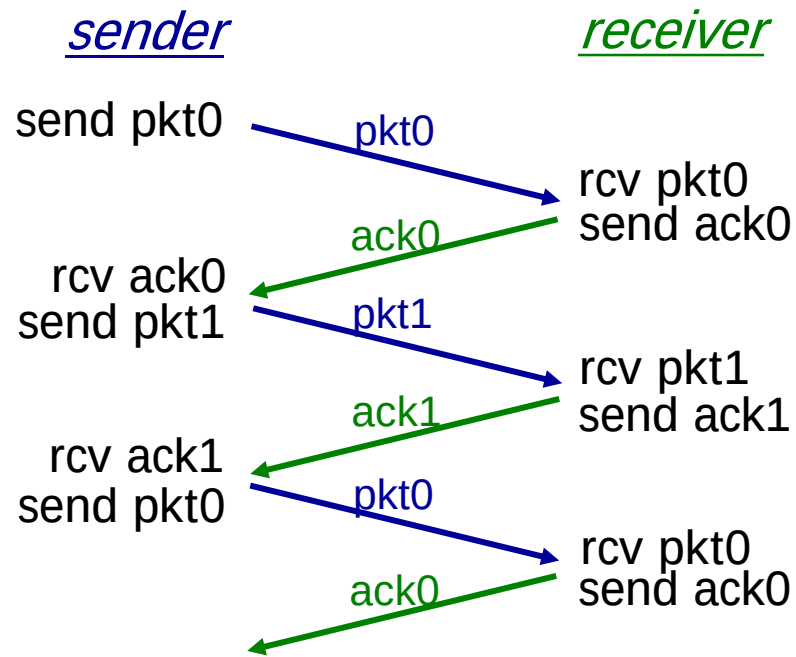


(a) no loss

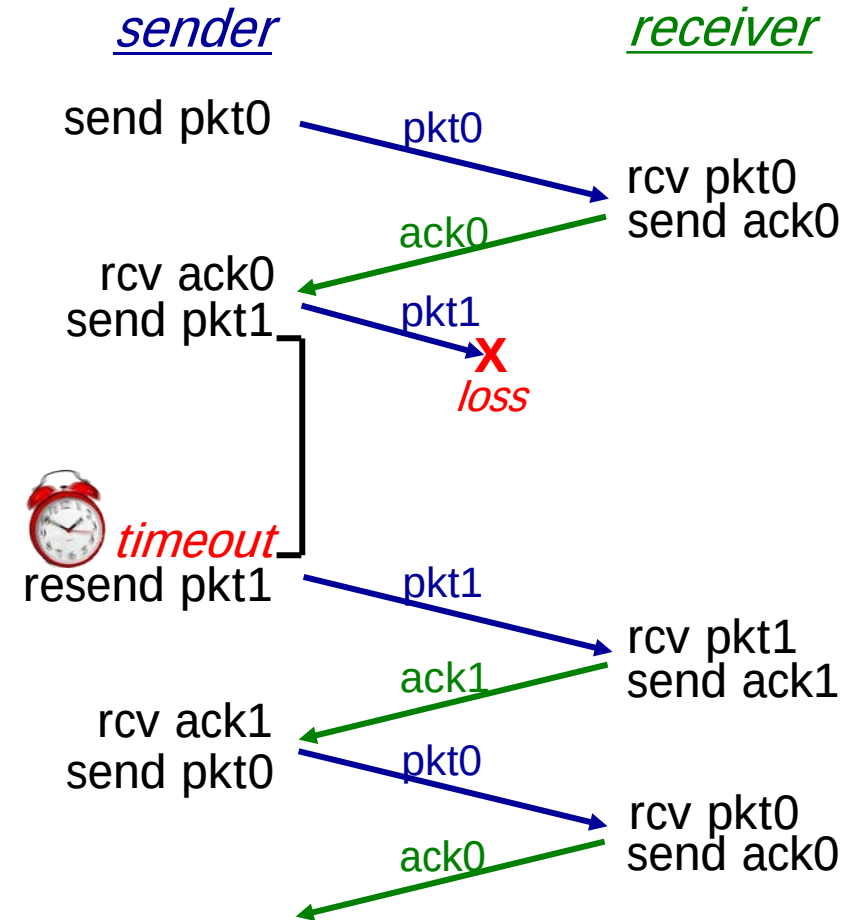
sender receiver

(b) packet loss

rdt3.0 in action

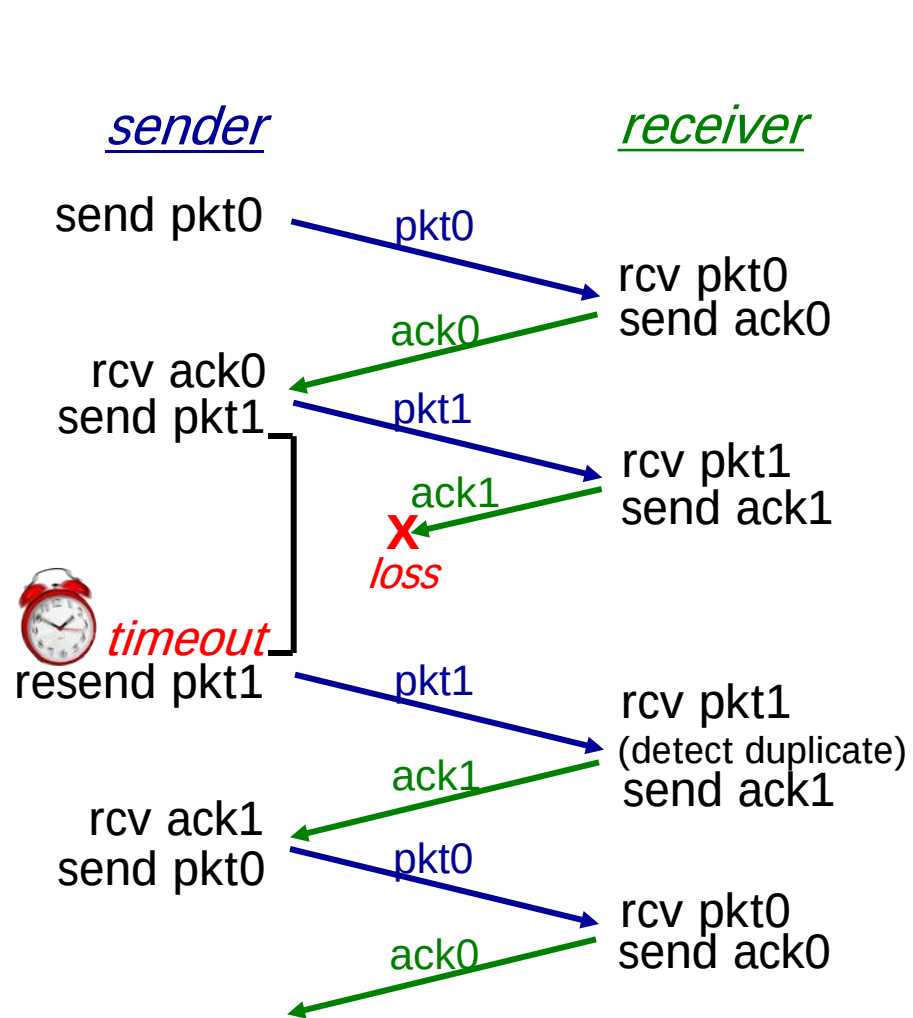


(a) no loss



(b) packet loss

rdt3.0 in action

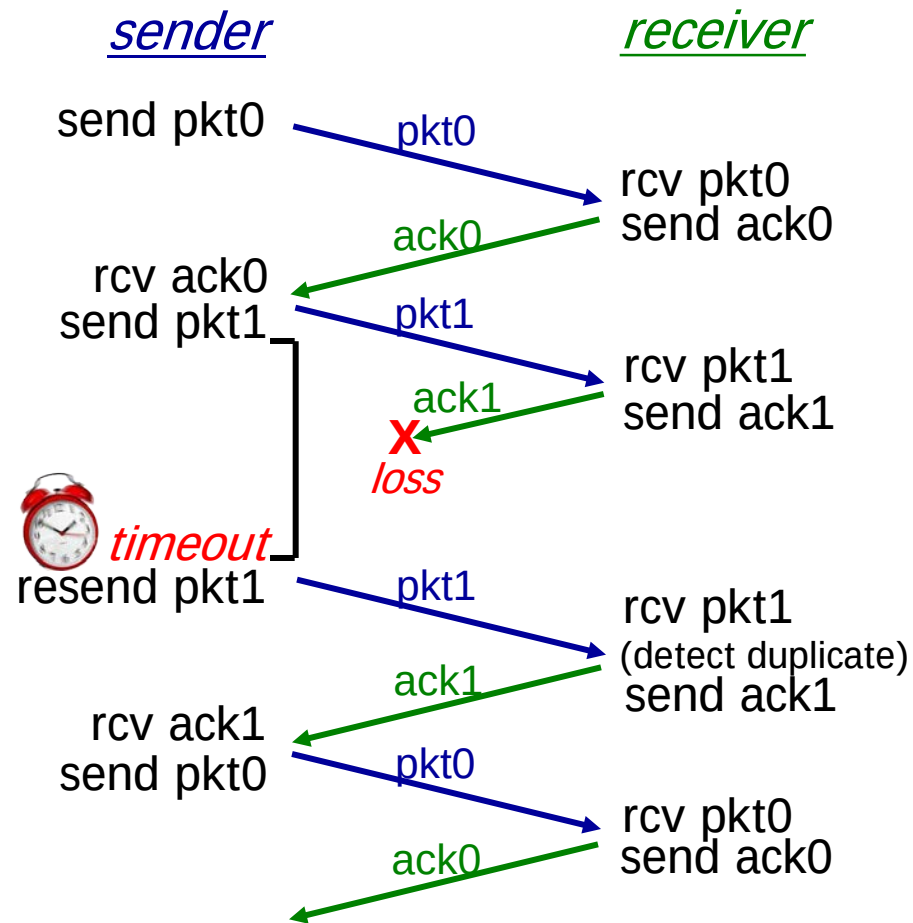


(c) ACK loss

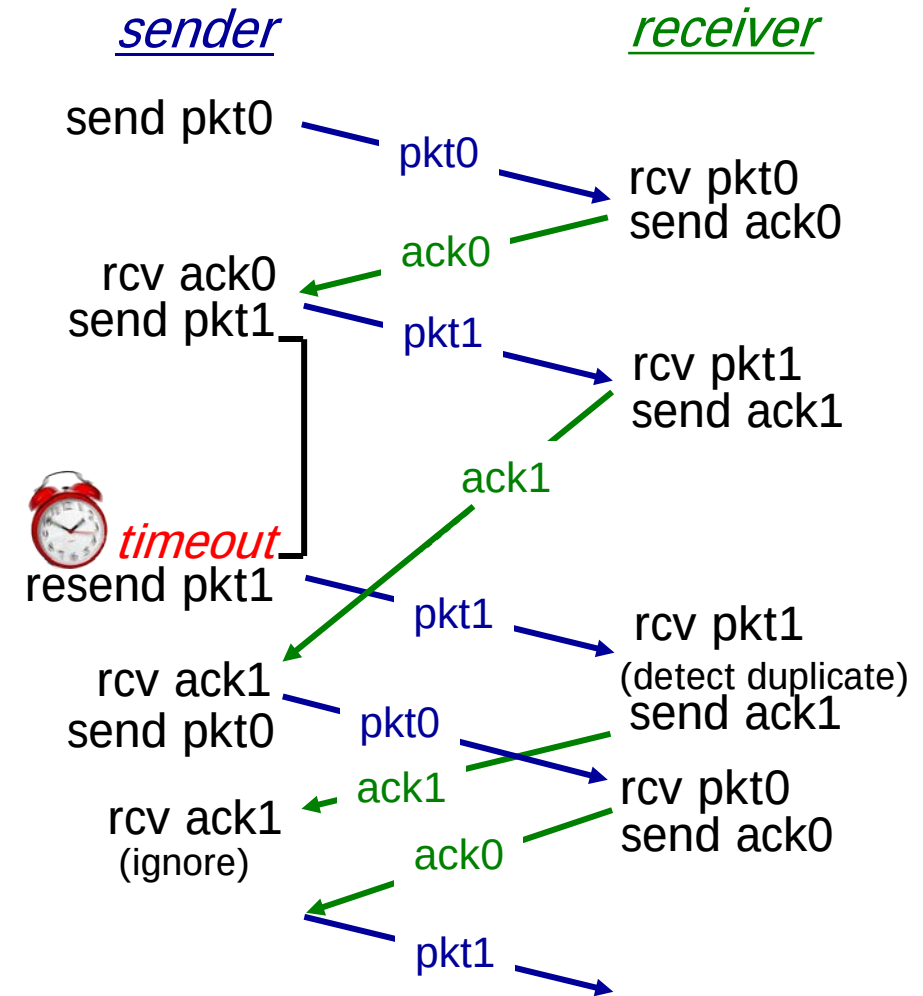


(d) premature timeout/ delayed ACK

rdt3.0 in action



(c) ACK loss



(d) premature timeout/ delayed ACK

Performance of rdt3.0 (stop-and-wait)

- U_{sender} : *utilization* – fraction of time sender busy sending

Performance of rdt3.0 (stop-and-wait)

- U_{sender} : *utilization* – fraction of time sender busy sending
- example: 1 Gbps link, 15 ms prop. delay, 8000 bit packet

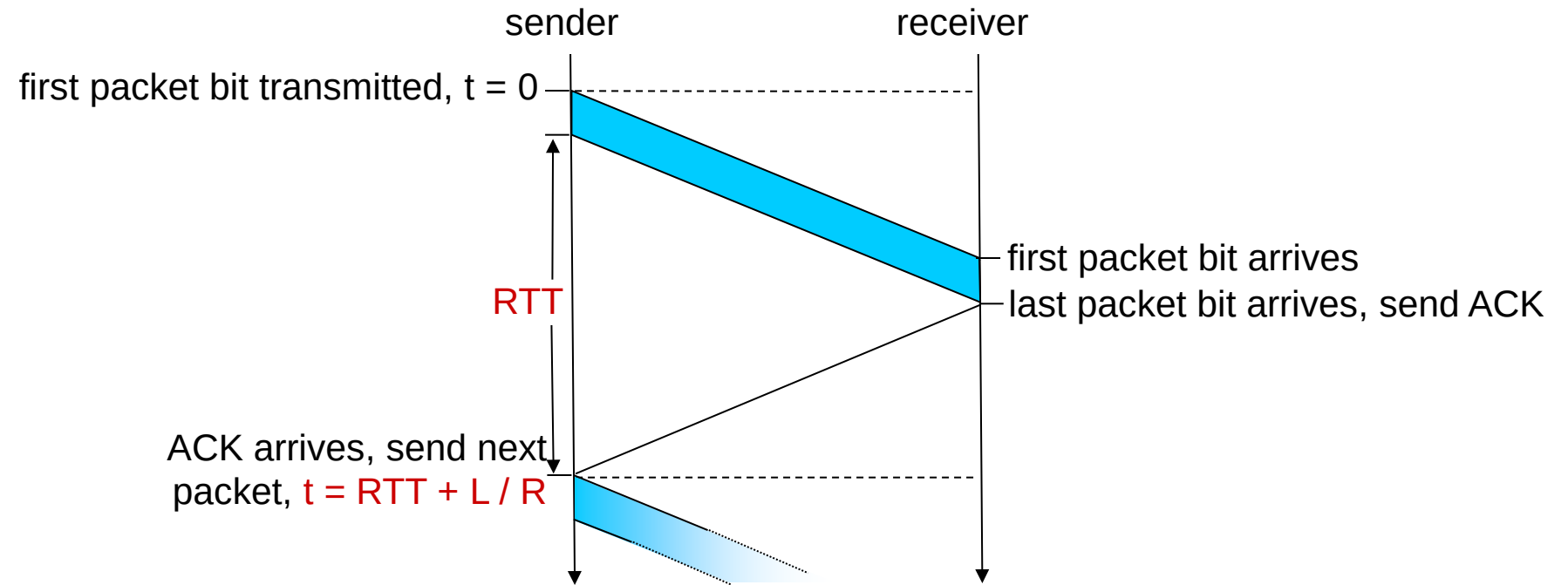
- time to transmit packet into channel:

$$D_{trans} = \frac{L}{R} = \frac{8000 \text{ bits}}{10^9 \text{ bits/sec}} = 0.008 \text{ ms}$$

- L : length of the message
- R : transmission rate

$$RTT = 2 D_{prop} = 2 \times 15 \text{ ms} = 30 \text{ ms}$$

rdt3.0: stop-and-wait operation



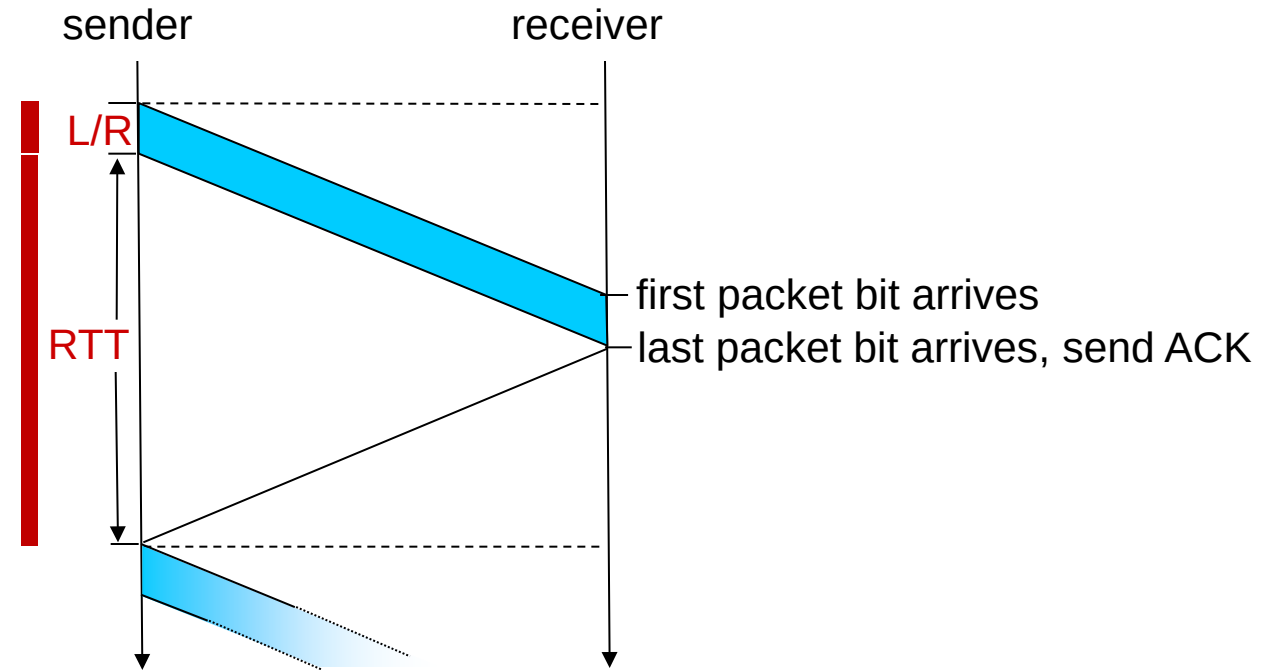
rdt3.0: stop-and-wait operation

- example: **1 Gbps link**, 15 ms prop. delay, 8000 bit packet

$$D_{trans} = \frac{L}{R} = 0.008 \text{ ms}$$

$$RTT = 2 D_{prop} = 30 \text{ ms}$$

$$U_{\text{sender}} = \frac{L / R}{RTT + L / R} = \frac{0.008}{30.008} = 0.00027$$

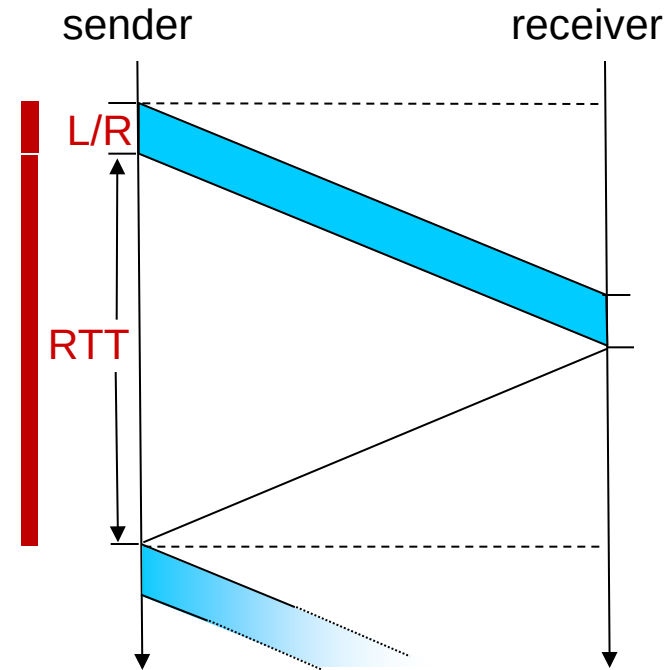


- The sender was busy only 0.027% of the time!
- The sender was able to send only 1,000 bytes in 30.008 ms
- Its effective throughput is **only 267 kbps** vs. **1 Gbps link was available!**

rdt3.0: stop-and-wait operation

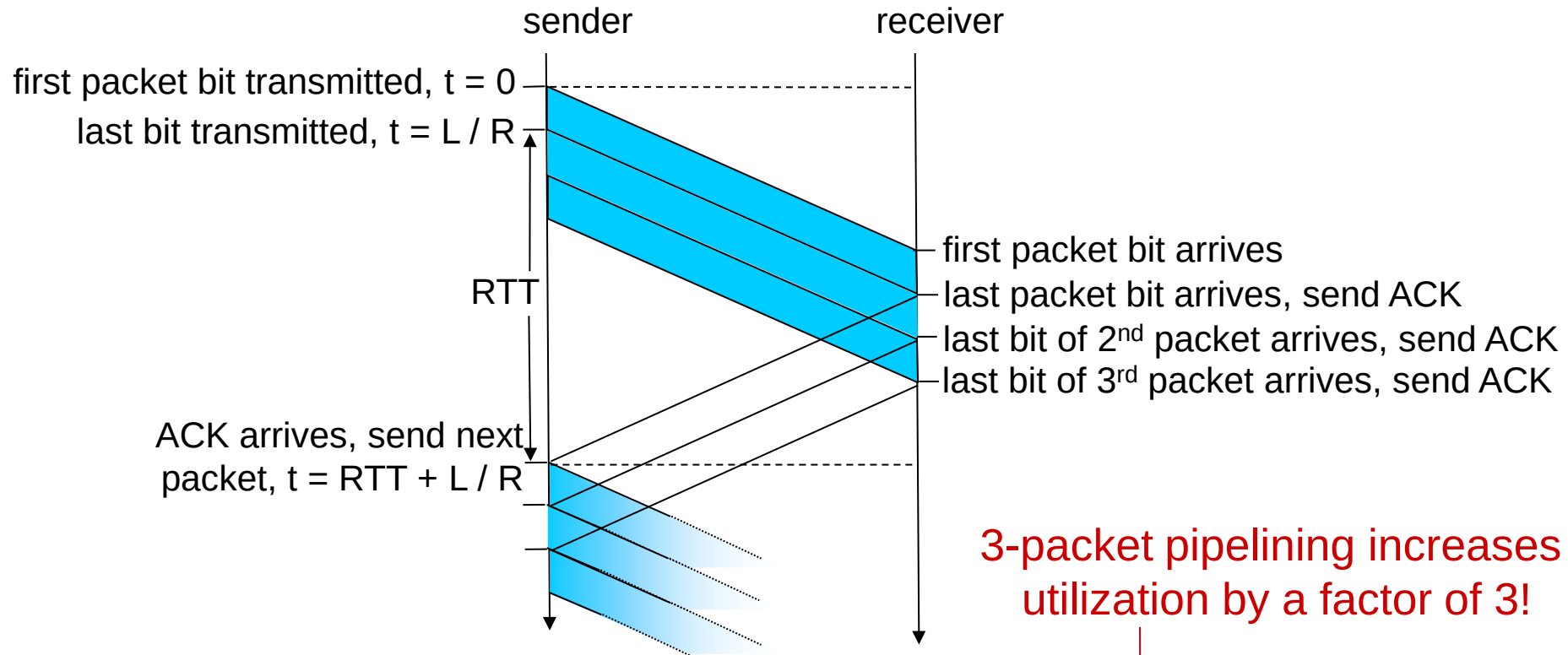
- rdt 3.0 performs bad in utilization due to the **stop-and-wait** operation
- Protocol limits performance of underlying infrastructure (channel)

How to fix this?



- **pipelining**: sender allows multiple, “in-flight”, yet-to-be-acknowledged packets

Pipelining: increased utilization



3-packet pipelining increases utilization by a factor of 3!

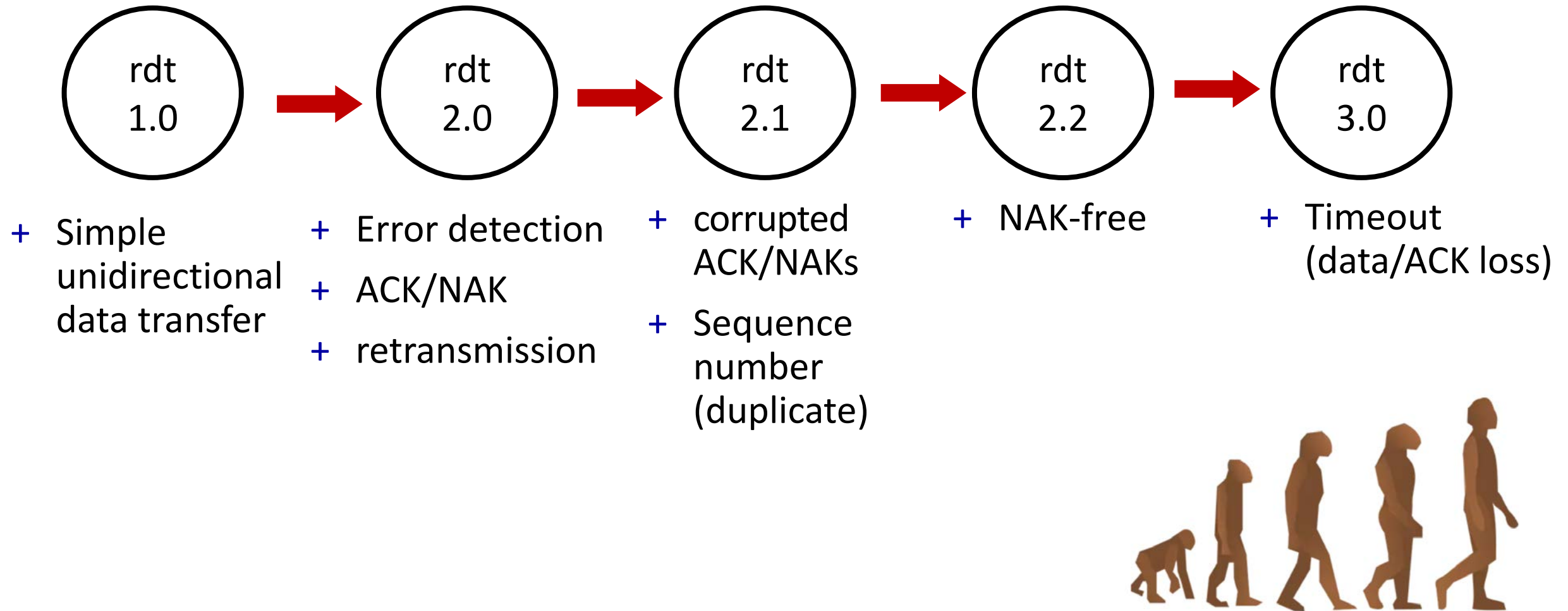
$$U_{\text{sender}} = \frac{3L / R}{RTT + L / R} = \frac{.0024}{30.008} = 0.00081$$

rdt3.0: pipelined protocols operation

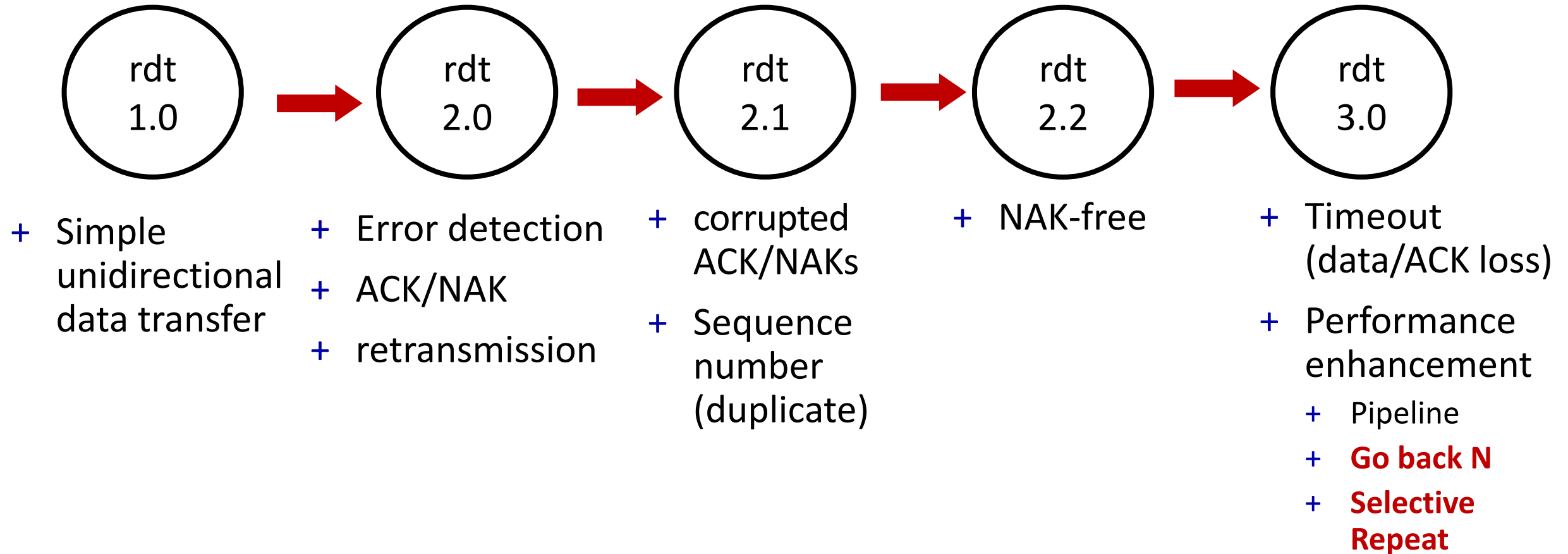
pipelining: sender allows multiple, “in-flight”, yet-to-be-acknowledged packets

- range of sequence numbers must be increased
- buffering at sender and/or receiver
- 2 basic approaches toward pipelined error recovery:
 - **Go-Back-N (GBN)**
 - **Selective repeat (SR)**

rdt evolution



rdt evolution



Chapter 3: roadmap

- Transport-layer services
- Multiplexing and demultiplexing
- Connectionless transport: UDP
- Principles of reliable data transfer (continued)
 - Go-Back-N & Selective Repeat
- Connection-oriented transport: TCP
- Principles of congestion control
- TCP congestion control
- Evolution of transport-layer functionality



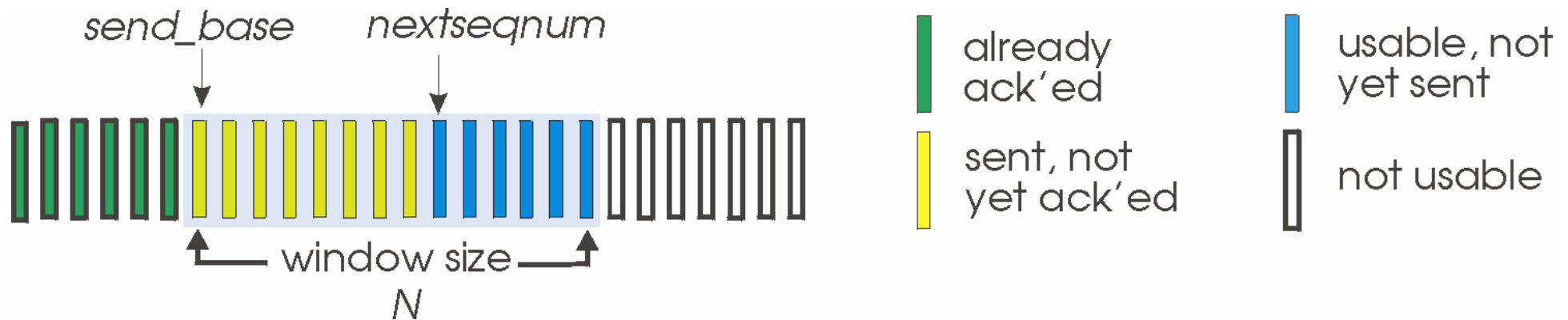
rdt3.0: pipelined protocols operation

pipelining: sender allows multiple, “in-flight”, yet-to-be-acknowledged packets

- range of sequence numbers must be increased
- buffering at sender and/or receiver
- 2 basic approaches toward pipelined error recovery:
 - **Go-Back-N (GBN)**
 - **Selective repeat (SR)**

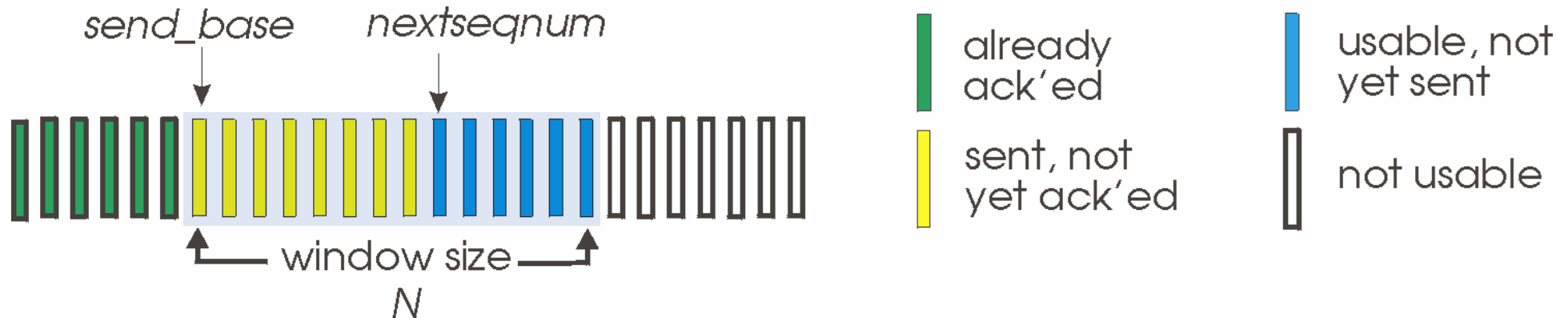
Go-Back-N: sender

- sender: “window” of up to N , consecutive transmitted but unACKed pkts
 - k -bit seq # in pkt header



Go-Back-N: sender

- sender: “window” of up to N , consecutive transmitted but unACKed pkts
 - k -bit seq # in pkt header

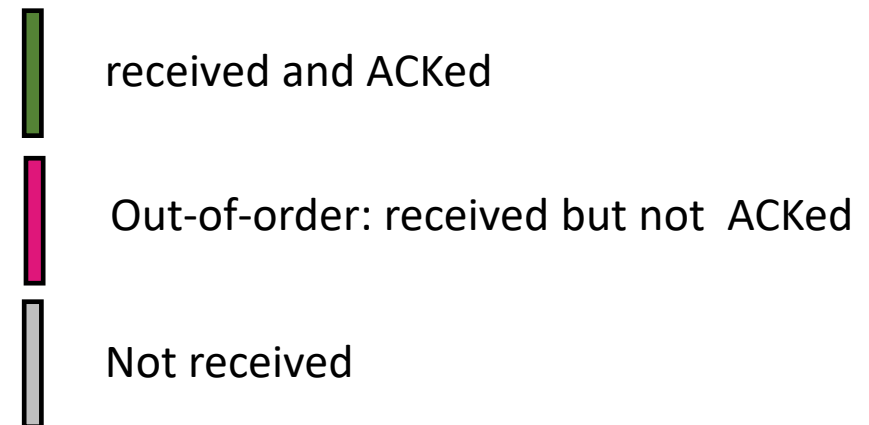
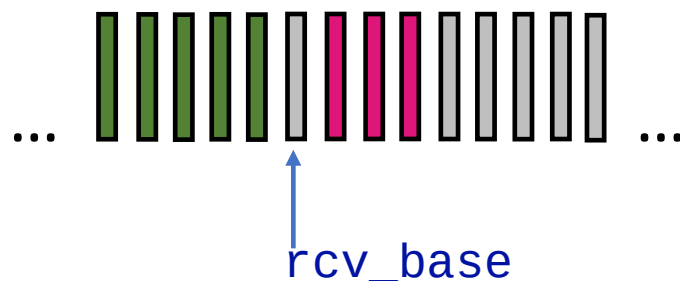


- **cumulative ACK:** $ACK(n)$: ACKs all packets up to & including seq # n
 - on receiving $ACK(n)$: move window forward to begin at $n+1$
- **timer** for oldest in-flight packet
- **timeout(n):** retransmit packet n and all higher seq # packets in window

Go-Back-N: receiver

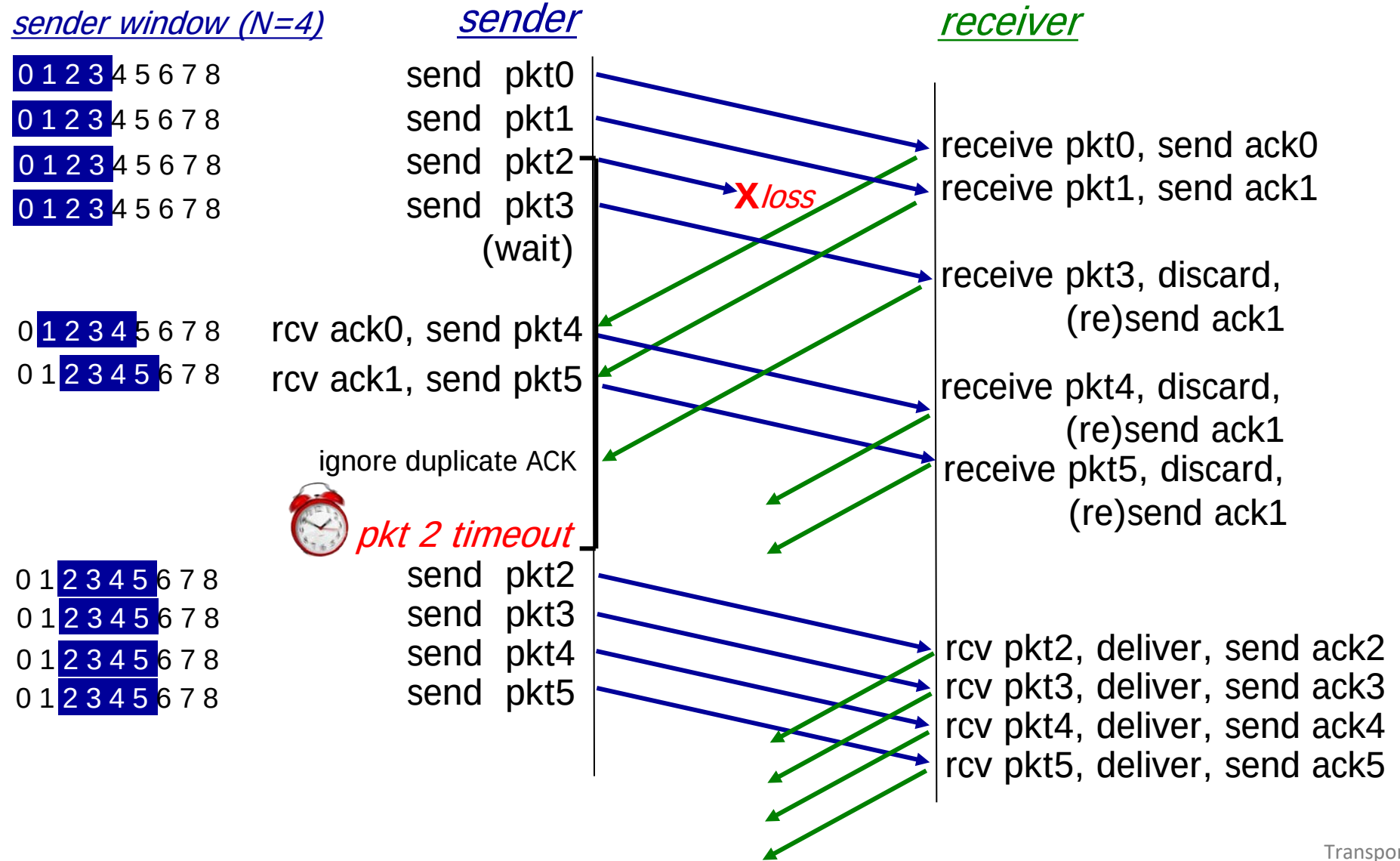
- ACK-only: **always and only** send ACK for correctly-received packet so far, with highest **in-order** seq #
 - may generate duplicate ACKs
 - need only remember **rcv_base**
- on receipt of out-of-order packet:
 - usually discard (don't buffer)
 - re-ACK pkt with highest in-order seq #

Receiver view of sequence number space:



Interactive animation: https://www2.tkn.tu-berlin.de/teaching/rn/animations/gbn_sr/

Go-Back-N in action



rdt3.0: pipelined protocols operation

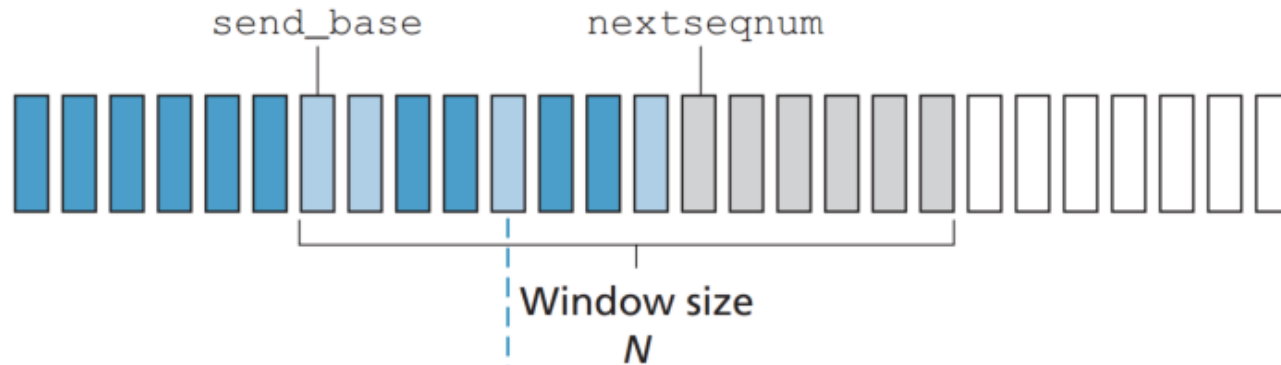
pipelining: sender allows multiple, “in-flight”, yet-to-be-acknowledged packets

- range of sequence numbers must be increased
- buffering at sender and/or receiver
- 2 basic approaches toward pipelined error recovery:
 - **Go-Back-N (GBN)**
 - **Single error performance problem:** a single packet error can cause GBN to retransmit a large number of packets, many unnecessarily
 - **Selective repeat (SR)**

Selective repeat: the approach

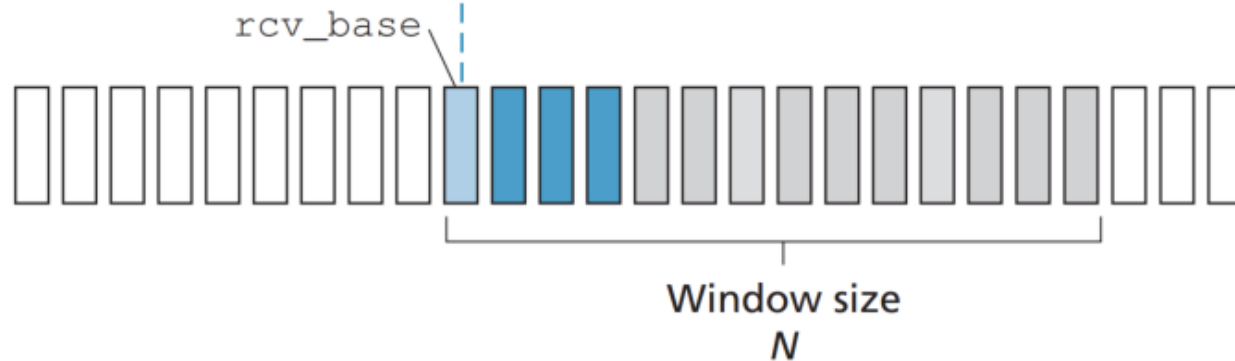
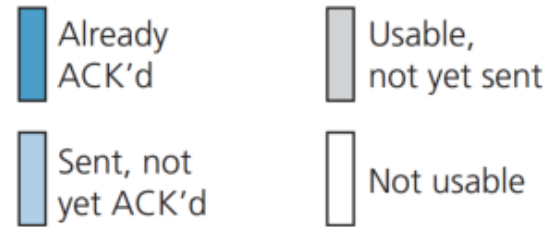
- *pipelining*: multiple packets in flight
- *receiver individually ACKs* all correctly received packets
 - buffers packets, as needed, for in-order delivery to upper layer
- *sender*:
 - maintains (conceptually) a **timer** for each unACKed pkt
 - timeout: retransmits single unACKed packet associated with timeout
 - maintains (conceptually) “**window**” over **N** consecutive seq #s
 - limits pipelined, “in flight” packets to be within this window

Selective repeat: sender and receiver



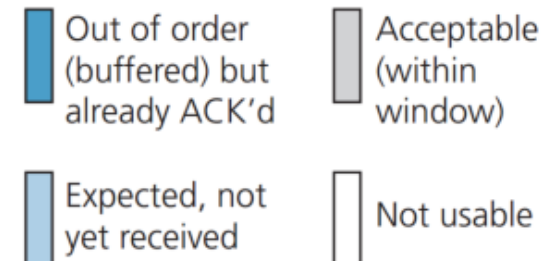
a. Sender view of sequence numbers

Key:



b. Receiver view of sequence numbers

Key:



Selective repeat: sender

data from above:

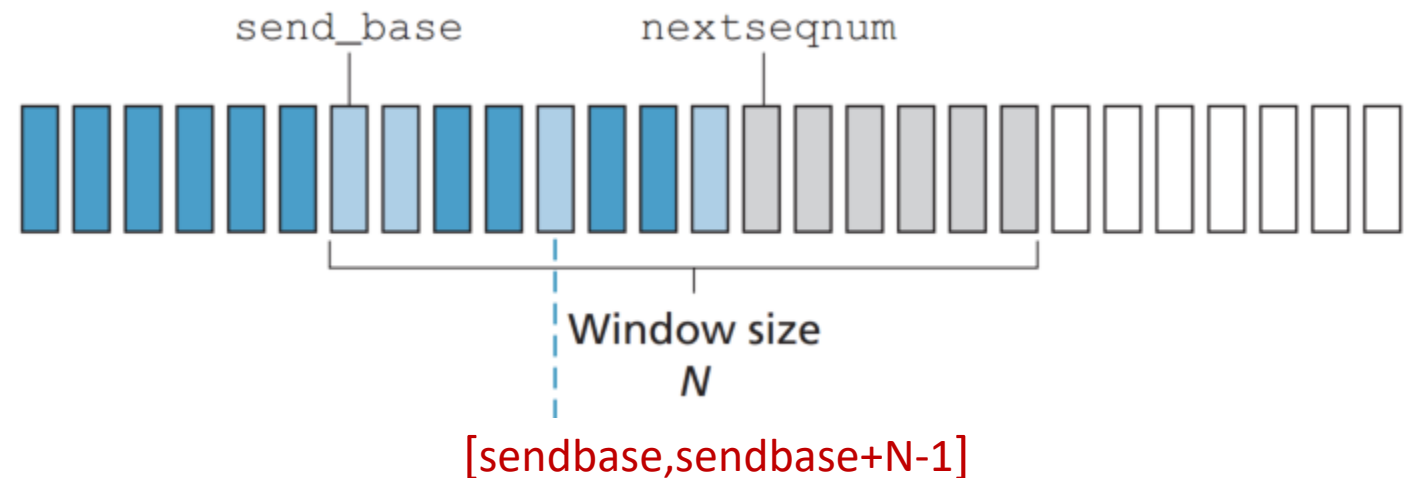
- if next available seq # in window, send packet

timeout(n):

- resend packet n , restart timer

ACK(n) in $[\text{sendbase}, \text{sendbase}+N-1]$:

- mark packet n as received
- if n smallest unACKed packet, advance window base to next unACKed seq #



Key:

Dark blue box: Already ACK'd

Light blue box: Sent, not yet ACK'd

Gray box: Usable, not yet sent

White box: Not usable

Selective repeat: receiver

packet n in $[rcvbase, rcvbase+N-1]$

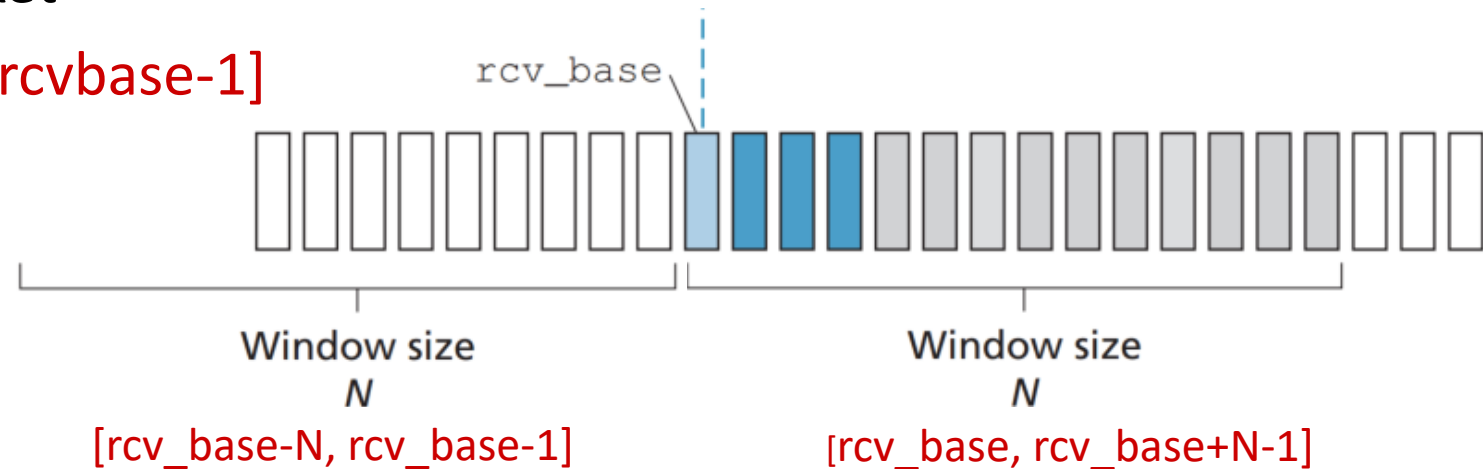
- send $ACK(n)$
- out-of-order: buffer
- in-order: deliver (also deliver buffered, in-order packets), advance window to next not-yet-received packet

packet n in $[rcvbase-N, rcvbase-1]$

- $ACK(n)$

otherwise:

- ignore



Key:

Already
ACK'd

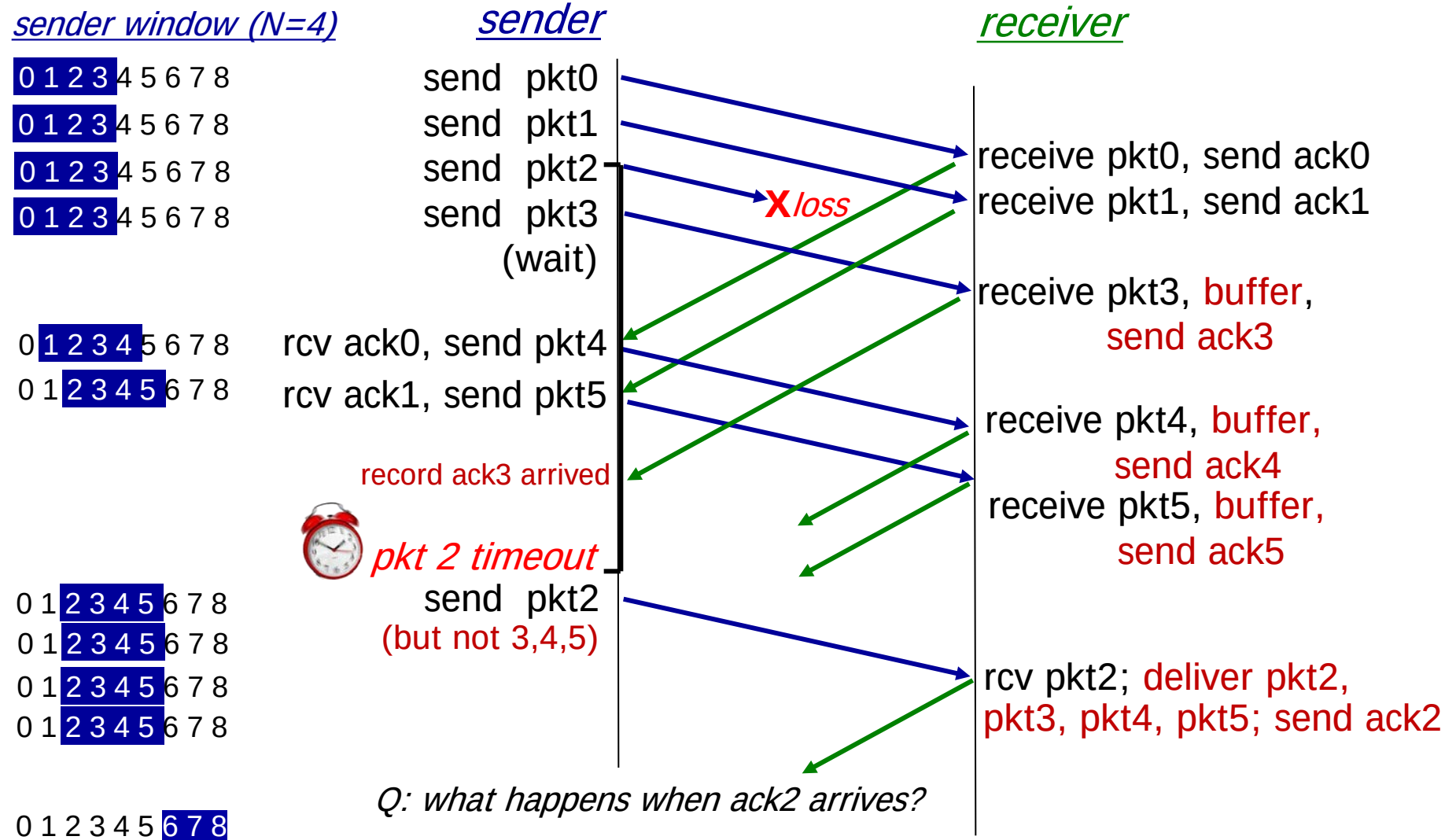
Usable,
not yet sent

Sent, not
yet ACK'd

Not usable

Transport Layer: 3-85

Selective Repeat in action

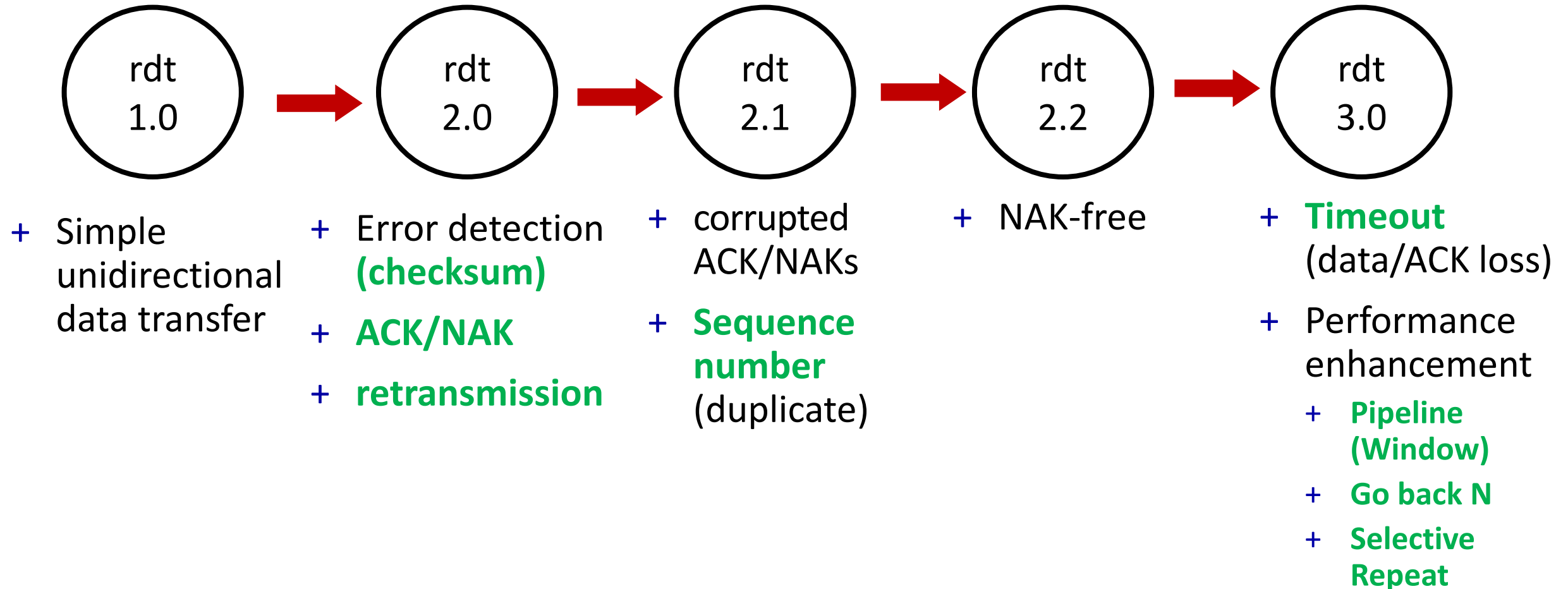


rdt3.0: pipelined protocols operation

pipelining: sender allows multiple, “in-flight”, yet-to-be-acknowledged packets

- range of sequence numbers must be increased
- buffering at sender and/or receiver
- 2 basic approaches toward pipelined error recovery:
 - **Go-Back-N (GBN)**
 - **Single error performance problem:** a single packet error can cause GBN to retransmit a large number of packets, many unnecessarily
 - **Selective repeat (SR)**
 - **The window size must be less than or equal to half the size of the sequence number space**

rdt evolution



Chapter 3: roadmap

- Transport-layer services
- Multiplexing and demultiplexing
- Connectionless transport: UDP
- Principles of reliable data transfer
- **Connection-oriented transport: TCP**
 - segment structure
 - reliable data transfer
 - flow control
 - connection management
- Principles of congestion control
- TCP congestion control

